

第 34 章: Exception Coding Details (异常编码细节)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本书"异常"部分的语法细节篇。第 33 章先让我们对异常相关语句有了快速印象, 本章则深入挖掘全部细节: try 的所有子句组合 (except/else/finally/except)、raise 的三种形式、assert 条件断言、以及 with 语句背后的上下文管理器协议。这一章是读者从"见过异常"走向"熟练驾驭异常"的关键一跃, 也是第 35 章(异常对象与类)的地基。

34.1 开篇引言 (Chapter Introduction)

英文原文

The prior chapter provided a quick look at exception-related statements in action. Here, we're going to dig a bit deeper—this chapter provides fuller coverage of exception-processing syntax in Python. Specifically, we'll explore the details behind the try, raise, assert, and with statements. Although these statements are mostly straightforward, you'll find that they offer power

rful tools for dealing with exceptional conditions in Python code.

中文翻译

前一章让我们快速见识了异常相关语句的实际应用。本章我们要挖得更深一点——本章将更完整地覆盖 Python 中异常处理 (exception-processing) 的语法。具体来说, 我们将探索 try、raise、assert 和 with 语句背后的细节。虽然这些语句大多直截了当, 但你会发现它们为处理 Python 代码中的异常状况提供了强大的工具。

深度理解

- 核心概念: 异常处理的四大语句——try (捕捉)、raise (主动引发)、assert (条件引发)、with (自动化进入/退出动作)。前 33 章里你可能已经零散见过它们, 本章把它们全部拼成完整拼图。

- 底层视角: "exception-processing syntax" 这个说法提醒我们: 异常不只是"错误报告机制", 而是一套控制流工具——异常发生时程序的跳转、匹配、传播路径都由解释器统一管理。

- 设计原因: Python 把异常当作"一等公民"的控制流结构 (而非像 C 语言那样只靠返回值判断错误), 好处是错误处理代码可以与正常逻辑分离, 不会污染主流程。

- 实际问题: 网络服务器、GUI 程序等需要"永不退出"的应用, 必须靠 try 把错误拦截在局部; 数据库事务、文件操作则需要 finally/with 保证清理动作必然执行。

·初学者误区：初学者常把异常当作"编程错误的标志"，但实际上异常是正常的语言机制——raise 可以主动引发自定义异常，assert 就是为调试而生的工具。学会"主动用异常"才是本章的目标。

34.2 The try Statement (try 语句)

英文原文

First up, the try statement is how your code catches exceptions. In short, if an exception occurs while running this statement's main block, the program jumps back to run one of the statement's handlers and continues from there. Its handlers may be specified by except, else, finally, and except clauses nested in the try, and separate rules apply to these clauses' syntax, and their valid combinations. This is a simple model on the surface, but the try statement's handler clauses have disjoint purposes, and its rules for valid combinations mean that it comes in distinct flavors. Because of this, we'll approach this subject by exploring the try's common roles in isolation first and putting their pieces together later as a combined statement. This parallels the fact that try really was separate statements in Python's dim past, but our focus here is on its unified present. Although technically part of the try, we'll also defer the except clause until the next chapter, partly because it encroaches on that chapter's exception-

object topic, but mostly because this is a tool that complicates the try story substantially for an extension that's rarely useful in practice. Our priority here is learning the fundamentals.

中文翻译

首先，try 语句就是你的代码捕捉异常的方式。简而言之，如果在运行该语句的主代码块（main block）时发生异常，程序会跳回来运行该语句的某个处理器（handler），然后从那里继续。它的处理器可以由嵌套在 try 里的 except、else、finally 和 except 子句（clause）指定，而这些子句的语法及其有效组合各自有不同的规则。这个模型表面上很简单，但 try 语句的各处理器子句目的各不相同，而“哪些组合合法”的规则意味着它有几种不同的形态（flavor）。因此，我们将分而治之：先单独探索 try 的各种常见角色，之后再把它们拼装成一个组合语句。这也与历史吻合——在 Python 遥远的过去，try 确实是彼此独立的语句，不过我们这里关注的是它今天统一的形态。虽然 except 子句技术上讲属于 try 的一部分，但我们还是把它推迟到下一章再讲：部分原因是它侵占了那一章“异常对象”的主题，但主要原因在于，这是一个会显著复杂化 try 故事的扩展，而它在实践中很少派上用场。我们的优先任务是学好基础。

深度理解

·核心概念：try 是“异常的中转站”——主块（main block）是你要守护的代码，各子句是守护方案：except 拦截、else 兜底无异常情形、finally 保证清理、except 处理异常组。

- 底层实现：解释器执行 try 时，会记录当前的程序上下文（帧信息、异常处理入口），一旦块内抛出异常，就沿"最近的异常处理器"链查找匹配项；这本质上是解释器内置的一张"跳转表"。
- 设计原因：把异常处理写成语句而非函数调用，是因为异常发生时栈上所有中间帧都要展开（unwind），语句式语法让编译器可以精确地在每个栈帧上挂接清理和匹配逻辑。
- 实际问题：一套统一的 try 语法取代了历史上"try/finally 与 try/except 是两条独立语句"的旧格局，让代码更简洁——这也是本书反复强调"统一现形"（unified present）的原因。
- 初学者误区：误以为 else、finally 只能二选一或顺序随意——实际上它们有严格的组合规则（try -> except -> else -> finally），违反会在运行前被语法检查（syntax error）拦下。

34.3 try Statement Clauses (try 语句的子句)

英文原文

When you write a try statement, a variety of clauses can appear after and below the try header. Table 34-1 summarizes all the possible forms as both reference and preview. We've already seen that except clauses

catch exceptions and finally clauses run on the way out. New here, else clauses run if no exceptions are encountered, and except clauses process exception groups and support all except forms except the empty.

Formally, a try must use at least one of the clauses in Table 34-1. There may be any number of except clauses, but you can code else only if there is at least one except, and there can be only one else and one finally. A finally can appear in the same statement as except and else, with ordering rules given later in this chapter.

Table 34-1. try statement clauses and forms | Clause form | Interpretation | |---|---| | except: | Catch all (or all other) exception types | | except name: | Catch a specific exception only | | except name as var: | Catch the listed exception and assign its instance | | except (name1, name2): | Catch any of the exceptions listed in a tuple | | except (name1, name2) as var: | Catch any listed exception and assign its instance | | else: | Run if no exceptions are raised in the try block | | finally: | Always perform this block on exit, exception or not | | except ... (非空 except 形式) | Catch multiple exceptions in a group (Chapter 35) | We'll explore the as var part available in some of Table 34-1's clauses in more detail when we meet the raise statement later in this chapter because it provides access to the object raised as an exception via var. Before all that, let's get started by examining the more common clauses of Table

中文翻译

当你编写 try 语句时，各种子句可以出现在 try 头部之后、下方。表 34-1 汇总了所有可能的形式，既作参考也作预览。我们已经见过 except 子句捕捉异常、finally 子句在退出时执行。这里的新内容是：else 子句在没有遇到异常时执行，except 子句则处理异常组（exception groups），支持除空 except 之外的所有 except 形式。形式上，一个 try 必须至少使用表 34-1 中的一个子句。except 子句可以有任意多个，但只有至少有一个 except 时你才能编写 else，而且 else 和 finally 各只能有一个。finally 可以与 except、else 出现在同一条语句中，其顺序规则将在本章后面给出。表 34-1. try 语句的子句与形式

子句形式	含义
except:	捕捉所有（或所有其他）异常类型
except name:	只捕捉特定异常
except name as var:	捕捉列出的异常，并把其实例赋给 var
except (name1, name2):	捕捉元组中列出的任一异常
except (name1, name2) as var:	捕捉任一系列出的异常，并把其实例赋给 var
else:	当 try 块中没有引发异常时运行
finally:	退出时总是执行此块，无论有无异常
except ...（非空 except 形式）	在组中捕捉多个异常（第 35 章）

表 34-1 中某些子句里的 as var 部分，我们将在本章后面遇到 raise 语句时再深入探讨，因为它通过 var 提供了对"作为异常被引发的对象"的访问。在那之前，让我们先更仔细地考察表 34-1 中更常见的子句。

深度理解

·核心概念：一张表看懂 try 的"菜单"——空 except（全能）、具名 except（点菜）、元组 except（多选）、as var（拿到异常实例）、else（无事发生）、finally（必然执行）、except（异常组，下章讲）。

·底层实现：except (A, B) as var 在字节码层面编译为"逐一比较异常类型（isinstance 检查），命中即绑定局部变量"；空 except 则是无条件命中，对应一个不检查就跳转的处理器。

·设计原因：允许"元组多选 + 同一处理器"是为了消除重复代码——多个异常共享同一个处理逻辑时，不必写多遍相同代码块。

·实际问题：形式规则（至少一个子句、else 依赖 except、只有一个 else/finally）保证了语句含义无歧义：如果没有 except，else 就没有"参照系"可言。

·初学者误区：把 except: 当作"万能药"到处用，或者记不住"else 必须搭配 except"。另注意 except 是 Python 3.11 才引入的新语法，老项目里看不到它——本书特意把它推迟到下一章，就是为了不让它干扰基础学习。

34.4 The except and else Clauses (except 与 else 子句)

英文原文

Syntactically, the try is a compound, multipart statement. It starts with a try header line, followed by a block of (usually) indented statements, which is followed by clauses that each identify a condition to be handled and give a block of statements to handle it. In its most common form, try is coded with one or more except clauses and an optional else clause at the end.

You associate the words try, except, and else by indenting them to the same level (i.e., lining them up vertically), like this:

```
python try: statements # 先运行这个主动作
except name1: statements # 若 try 块中引发 name1 则运行
except (name2, name3): statements # 若 try 中发生 name2 或 name3 则运行
except name4 as var: statements # 若引发 name4 则运行, 并将其赋给 var
except: statements # 为所有其他引发的异常运行
else: statements # 若 try 块中未引发异常则运行
```

Semantically, the block under the try header in this statement represents the main action of the statement—the code you're trying to run, and wrapping in exception handlers. The rest of the statement defines the handlers themselves: except clauses give handlers f

or exceptions raised during the try block, and the optional else clause gives a handler run if no exceptions occur in the try block.

Within a try, each except names exceptions to catch: a single exception catches just that exception, a tuple catches any exception in the tuple, and an except that omits the exception altogether matches all (or all other) exceptions. Each nonempty except can also give a variable name after as to be assigned the exception object raised by Python or raise statements; again, we'll explore this option ahead.

中文翻译

从语法上看，try 是一个复合的、多部分语句（compound, multipart statement）。它以 try 头部行开头，后面跟着一个（通常缩进的）语句块，再后面跟着若干子句——每个子句标识一种需要处理的条件，并给出处理它的语句块。在它最常见的形式中，try 编码一个或多个 except 子句，以及末尾一个可选的 else 子句。你要通过把 try、except、else 这些词缩进到同一层级（也就是在垂直方向上对齐它们）来把它们关联起来，就像这样：

```
python try: statements # 先运行这个主动作
except name1: statements # 若 try 块中引发 name1 则运行
except (name2, name3): statements # 若 try 中发生 name2 或 name3 则运行
except name4 as var: statements # 若引发 name4 则运行，并将其赋给 var
except: statements # 为所有其他引发的异常运行
else: statements # 若 try 块中未引发异常则运行
```

从语义上讲，本语句中 try 头部之下的代码块代表语句的

主动作 (main action) ——也就是你试图运行、并用异常处理器包裹起来的代码。语句的其余部分定义处理器本身：`except` 子句为 `try` 块运行期间引发的异常提供处理器，而可选的 `else` 子句则提供一个“如果 `try` 块中没有异常发生时运行的处理器。在一个 `try` 中，每个 `except` 指明要捕捉的异常：一个单个异常只捕捉那一个异常；元组捕捉元组中的任一异常；而完全省略异常的 `except` 匹配所有（或所有其他）异常。每个非空 `except` 还可以在 `as` 后面给一个变量名，用于接收 Python 或 `raise` 语句引发的异常对象；再说一次，这个选项我们稍后再深入。

深度理解

- 核心概念：`try` 的主块 = “被守护的代码”；`except` = 处理清单（自上而下匹配）；`else` = “平安无事”时的成功处理分支。缩进不是装饰，而是归属关系的语法。

- 底层实现：CPython 在编译含 `try` 的代码块时，会在字节码里插入 `SETUPEXCEPT/SETUPFINALLY` 等栈帧指令，异常发生时解释器调用 `PyErrExceptionMatches` 逐条做子类匹配；命中后执行处理器代码块。

- 设计原因：用“从上到下、逐一匹配”的线性规则，而不是像 C++/Java 那样只按类型精确匹配，是因为 Python 的异常是类层次结构——一个 `except Exception` 要能接住所有子类，线性扫描 + `isinstance` 语义最自然。

- 实际问题：如果你需要“任何失败都要记录日志、但只有特定错误才恢复”，这一节的结构就是模板：特定 `except` 放前面，通用兜底放最后，`else` 放成功分支。

·初学者误区：以为 `except` 匹配是"精确类型相等"——其实是"`isinstance` 关系"（子类也能匹配父类）；还容易忘记 `except` 一旦命中就不再继续往下试，所以顺序很重要：具体异常必须写在通用异常前面。

代码分析

```
try:
    statements                                #
except name1:
    statements                                # try name1
except (name2, name3):
    statements                                # try name2 name3 ...
except name4 as var:
    statements                                # name4 var
except:
    statements                                #
else:
    statements                                # try
```

·解释器如何处理：解释器先完整执行 `try` 块；若中途抛出异常，它立即中断当前执行，取出异常的类对象，与第一个 `except` 后面的名字做 `isinstance` 匹配——不匹配就试下一个，匹配就执行对应处理器（有 `as var` 则先绑定异常实例）。`else` 块只在 `try` 块正常跑完（没有异常）后才执行。注意：`else` 不会因为"异常被捕捉过"而执行——"捕捉过"意味着有异常，属于 `except` 的功劳。

·设计思想：这个骨架展示了"具体 → 通用"的匹配次序：越精确的处理器越靠前，`except:` 兜底永远在最后。`as var` 让处理器能"拿到证据"（异常实例），这是后续调试与重抛（`reraise`）的入口。

34.4.1 How try Statements Work (try 语句如何运行)

英文原文

Operationally, here's how try statements are run. When a try statement is entered, Python records the current program context so it can return to it if an exception occurs. The statements nested under the try header are run first. What happens next depends on whether an exception is raised while the try block's statements are running, and whether a raised exception matches any of those that the try is watching for: **Exception and match** — If an exception occurs while the try block's statements are running, and the exception matches one that the statement names, Python jumps back to the try and runs the statements under its top most except clause that matches the raised exception, after assigning the raised exception object to the variable named by `as` in the clause (if present). After the except block runs, control resumes below the entire try statement. If the except block itself raises another exception, the propagation process is started anew from this point in the code. **Exception and no match** — If an exception occurs while the try block's statements are running, but the exception does not match one that the statement names, the exception is propagated up to the next most recently entered try statement.

t that matches the exception; if no such matching try statement can be found and the search reaches the top level of the program, Python prints a default error message and terminates the program (unless it's the REPL). No exception — If an exception does not occur while the try block's statements are running, Python runs the statements under the else clause (if present), and control then resumes below the entire try statement. If the else block itself raises another exception, it kicks off the propagation process again. In sum, except clauses catch any matching exceptions that happen while the try block is running, and the else clause runs only if no exceptions happen while the try block runs. Exceptions raised are matched to exceptions named in except clauses by class relationships we'll explore both ahead and in the next chapter (brief: a subclass matches its superclass), and the empty except clause with no exception name matches any exception. In effect, except clauses are focused exception handlers—they catch exceptions that occur only within the statements in the associated try block. However, as the try block's statements can call functions coded elsewhere in a program, the source of an exception may very well be outside the code of the try statement itself. In fact, a try block might invoke arbitrarily large amounts of program code—including code that may have try statements of its own, which will be searched first when exceptions occur. In other words, be

cause try statements can nest at runtime, where an exception goes depends on the code run before it, a phenomenon we'll explore in Chapter 36. If a finally clause is added to a try, its code block is run for all three of the cases listed previously, as you'll see ahead. First, though, let's take a look at some common variations of exception-catching clauses.

中文翻译

从运行机制上讲，try 语句是这样运行的。当进入一条 try 语句时，Python 会记录当前的程序上下文（context），以便在发生异常时能返回此处。try 头部下面嵌套的语句先运行。接下来发生什么，取决于 try 块的语句运行期间是否引发了异常，以及引发的异常是否与 try 正在监视的某个异常匹配：异常且匹配——如果 try 块的语句运行期间发生异常，且异常匹配语句点名的某个异常，Python 会跳回 try，先按 as（若有）把引发的异常对象赋给子句中的变量，然后运行与引发的异常匹配的最靠上（topmost）那个 except 子句下的语句。except 块运行完毕后，控制流在整条 try 语句下方继续。如果 except 块自己又引发了另一个异常，传播过程将从代码中的这一点重新开始。异常但不匹配——如果 try 块的语句运行期间发生异常，但异常不匹配语句点名的任何一个，该异常就被传播（propagate）到下一个“最近进入的、且匹配该异常”的 try 语句；如果找不到这样的 try 语句、搜索到达程序顶层，Python 会打印一条默认错误信息并终止程序（除非是在 REPL 里）。无异常——如果 try 块的语句运行期间没有发生异常，Python 会运行 else 子句（若有）下的语句，然后控制流在整条 try 语句下

方继续。如果 `else` 块自己引发了另一个异常，它会再次启动传播过程。总而言之：`except` 子句捕捉 `try` 块运行期间发生的任何匹配异常；`else` 子句只在 `try` 块运行期间没有异常时运行。被引发的异常是按类关系与 `except` 子句点名的异常进行匹配的——我们稍后和下一章都会深入（简短版：子类匹配它的超类）；而没有任何异常名的空 `except` 子句匹配任何异常。实际上，`except` 子句是聚焦的（*focused*）异常处理器——它们只捕捉关联 `try` 块中语句内部发生的异常。不过，由于 `try` 块的语句可以调用程序中别处编写的函数，异常的来源很可能在 `try` 语句自身的代码之外。事实上，一个 `try` 块可能调用任意大量的程序代码——包括那些自身带有 `try` 语句的代码，这些代码里的 `try` 会在异常发生时优先被搜索。换句话说，由于 `try` 语句可以在运行时嵌套，异常去哪里取决于它之前运行的代码——这一现象我们将在第 36 章探索。如果给 `try` 加上 `finally` 子句，它的代码块对前面列出的三种情况都会运行，你稍后会看到。不过首先，让我们看看捕捉异常的子句的一些常见变体。

深度理解

·核心概念：三条运行规则——匹配则跳回执行 `except` 并从 `try` 之后继续；不匹配则向上传播（交给外层 `try` 或顶层默认处理）；无异常则执行 `else`。加上“处理器自己出错会重启传播”，这就是异常的全部动力学。

·底层实现：CPython 解释器维护一条“异常处理器链”。每个 `try` 块进入时把处理器信息压栈，异常发生时从链顶（最近进入的 `try`）开始搜索；找不到匹配就一直弹栈，直到顶层由 `sys.excepthook` 打印 `traceback` 并退出。

·设计原因："传播 + 最近优先"的设计让异常能自动穿过任意深的调用栈——`crashware.py` 里三层函数调用，异常却能一路冒到顶层，这正是"异常隔离业务代码"价值的体现。

·实际问题：理解这条规则，你就明白为什么"顶层脚本"不需要处理异常（报错即退出往往是对的），而"长期运行的服务器"必须在每次请求边界设 `try`（否则一个请求的异常会杀死整个进程）。

·初学者误区：误以为"`try` 块内函数的异常不会冒出来"——恰恰相反，异常会穿透所有中间函数，除非沿途某处有匹配的 `except`。还有人不明白"异常被捕捉后就是"死的"（`dead`）"：一旦 `except` 处理完，异常就消失了，控制流绝不会回到出错点。

34.4.2 Catching Many Exceptions with a Tuple（用元组捕捉多个异常）

英文原文

Per the fourth and fifth entries in Table 34-1, `except` clauses that list many exceptions in a parenthesized tuple catch any of the listed exceptions. Because Python looks for a match within a given `try` by inspecting the `except` clauses from top to bottom, the tuple version has the same effect as listing each exception in its own `except` clause, but you have to code the common statement body associated with each only once. H

ere's a partial example of multiple except clauses at work, which demos just how specific your handlers can be: python try: ... except NameError: ... except IndexError: ... except (AttributeError, TypeError, SyntaxError): ... If an exception is raised while this try block is running, Python returns to the try and searches for the first except that names the exception raised. It inspects clauses from top to bottom—and left to right along the way—and runs the statements under the first clause that matches. If none match, the exception is propagated past this try. Note that parentheses are required around the tuple in the "any" form, and using an as in this form lets you check which exception occurred when you listed many: python try: ... except (AttributeError, TypeError, SyntaxError) as What: ... # 然后检查 What To learn more about both as, as well as what happens when no except matches, we must move on.

中文翻译

按表 34-1 的第四、第五项, except 子句在带括号的元组 (parenthesized tuple) 中列出多个异常时, 会捕捉所列异常中的任意一个。由于 Python 在一个 try 内查找匹配时是自上而下逐个检查 except 子句的, 元组版本与"把每个异常各写一个 except 子句"效果相同, 只是你只需把公共的语句体编写一次。这里是一个多个 except 子句发挥作用的局部示例, 它演示了你的处理器可以多么具体: python try: ... except NameError: ... except IndexError: ... except (AttributeError, TypeError, SyntaxError): ... 如果这个 t

try 块运行期间引发了异常，Python 会回到 try 并搜索第一个点名该异常类型的 except。它自上而下（沿途还从左到右）检查子句，运行第一个匹配子句下的语句。如果都不匹配，异常就被传播越过这个 try。注意，"任意一个"形式中元组周围的圆括号是必需的；在这种列出多个异常的形式里使用 as，可以让你检查究竟是哪一个异常发生了：python try: ... except (AttributeError, TypeError, SyntaxError) as What: ... # 然后检查 What 要了解更多关于 as 的细节，以及"没有 except 匹配时会发生什么"，我们必须继续往下读。

深度理解

- 核心概念：元组 except (A, B, C) 是"一份多买"——三种异常共享同一处理器，等价于三个各自成段的 except，但代码只写一份。
- 底层实现：匹配时解释器从左到右遍历元组中的每个类型做 isinstance 检查——这就是"从左到右"的来源；一旦命中，后面不再检查。
- 设计原因：同一个错误处理逻辑常常适用于一组相关异常（比如 (ValueError, TypeError) 都是"输入不合规"），元组语法避免了复制粘贴处理器体。
- 实际问题：真实代码里最常见的写法是 except (ValueError, TypeError) as e: 处理解析错误；注意元组必须加括号，写成 except ValueError, TypeError: 是旧版 Python 2 语法，Python 3 里会变成语法错误。
- 初学者误区：有人以为元组里的异常会"按顺序逐个尝试

处理"——不会，元组只选一个命中者，整个处理器只运行一次。另一个坑：圆括号不能省。

代码分析

```
try:
    ...
except NameError:
    ...
except IndexError:
    ...
except (AttributeError, TypeError, SyntaxError) as What:
    ...
```

·解释器如何处理：异常抛出后，解释器把异常类依次与 `NameError`、`IndexError` 比较；都失败后，再与元组内的 `AttributeError`、`TypeError`、`SyntaxError` 逐个比较，命中即绑定 `What` 并执行处理器。若三类都不匹配，异常继续向上传播。

·设计思想：这个示例刻意展示"具体在前、聚合在后"的层次感。`as What` 相当于给处理器一个"现场证人"——你可以用 `isinstance(What, TypeError)` 之类的判断走更细的分支，而不必拆开写多条 `except`。

34.4.3 Catching All Exceptions with `Emptyes` and `Exception` (用空 `except` 与 `Exception` 捕捉所有异常)

英文原文

Per the first entry in Table 34-1, except clauses that list no exception name catch all exceptions not previously listed in the try statement. That is, if you want to code a general "catchall" handler to be run when no other except clause matches the exception raised, an empty except does the trick: `python try: ... except NameError: ... # 处理 NameError except IndexError: ... # 处理 IndexError except: ... # 处理所有其他异常else: ... # 处理无异常的情况 (预览)` The empty except clause is a sort of wildcard feature—because it catches everything, adding it to the mix allows your handlers to be as general or specific as you like. In some scenarios, this form may be more convenient than listing all possible exceptions in a try—especially when working interactively in a REPL, or writing code that must recover no matter what occurs. For example, the following catches everything by not listing anything: `python try: ... except: ... # 捕捉所有可能的异常` That being shown, the empty except can also cause problems. It may catch unexpected exceptions, and intercept events unrelated to your code and required by another handler. For example, even system exit calls and Ctrl+C key-combination interrupts in Python work by triggering exceptions, and you usually want these to pass. Perhaps worse, the empty except may also catch genuine programming mistakes for which you probably want to

see an error message. Otherwise, you may not even know that a bug exists until it's too late to avoid a user's report. We'll revisit this as a gotcha at the end of this part of the book. For now, the standard "use with care" applies. Python provides an alternative that solves at least one of these problems—catching the built-in `Exception` has almost the same effect as an empty `except`, but won't catch exceptions related to system exits and `Ctrl+C`: `python try: ... except Exception: ...`

捕捉所有可能的异常——但除了退出类 We'll explore how this form does its magic formally in the next chapter when we study exception classes. In short, it works because exceptions match if they are a subclass of one named in an `except` clause, and `Exception` is a superclass of all the exceptions you should generally catch this way. This form has most of the same convenience of the empty `except` without the risk of catching exit events and also allows you to check the exception raised via `as`. While better, though, it also has some of the same risks—it may still mask and silently ignore programming errors. The opening snippet of this section deliberately listed an `else` clause, to call out that it is not a catchall like the empty `except`—an understandable source of confusion for try newcomers. The next section dissects the difference.

中文翻译

按表 34-1 的第一项，没有列出任何异常名的 `except` 子句，会捕捉 `try` 语句中此前未列出的所有异常。也就是说，如果你想编写一个通用的“捕获一切”（catchall）处理器，在没有任何其他 `except` 子句匹配到所引发异常时运行它，一个空 `except` 就能搞定：`python try: ... except NameError: ... # 处理 NameError except IndexError: ... # 处理 IndexError except: ... # 处理所有其他异常else: ... # 处理无异常的情况（预览）`空 `except` 子句是一种通配符（wildcard）特性——因为它捕捉一切，把它加入组合后，你的处理器想要多通用或多具体都可以。在某些场景下，这种形式比在 `try` 中列出所有可能的异常更方便——尤其在 REPL 中交互式工作时，或者编写“无论发生什么都必须恢复”的代码时。例如，下面这段通过什么都不列来捕捉一切：`python try: ... except: ... # 捕捉所有可能的异常`话虽如此，空 `except` 也可能引发问题。它可能捕捉到意料之外的异常，并且拦截那些与你的代码无关、却被另一个处理器需要的系统事件。例如，在 Python 中就连系统退出（`system exit`）调用和 `Ctrl+C` 组合键中断也是靠触发异常来工作的，而你通常希望它们放行通过。也许更糟的是，空 `except` 还可能捕捉到真正的编程错误——对这种错误你大概是想看到报错信息的。否则，你可能直到“已经来不及避免用户报 bug”时才意识到存在缺陷。我们会在本书这一部分的结尾把这一点作为“陷阱”（gotcha）重新审视。眼下，标准的“慎用”忠告适用。Python 提供了一个至少能解决其中一个问题的替代方案——捕捉内置的 `Exception` 与空 `except` 效果几乎相同，但不会捕捉与系统退出和 `Ctrl+C` 相关的异常：`python try: ... except Exception: ... # 捕捉所有可能的异常`——但除了退出类我们将在下一章学习异常类时正式探讨这种形式

是如何“施魔法”的。简而言之，它之所以有效，是因为：只要异常是 `except` 子句所点名的某个异常的子类，就视为匹配；而 `Exception` 是“你应该用这种方式捕捉”的所有异常的超类。这种形式拥有空 `except` 的大部分便利，却没有捕捉退出事件的风险，而且还能通过 `as` 检查被引发的异常。不过，虽然它更好，它也有一些同样的风险——它仍可能掩盖并静默忽略编程错误。本节开头的代码片段故意列出了一个 `else` 子句，是为了指出：它不是像空 `except` 那样的全能捕捉器——这是 `try` 新手一个可以理解的困惑来源。下一节将剖析这个区别。

深度理解

- 核心概念：三种“全能”级别——空 `except`: (连 `SystemExit`、`KeyboardInterrupt` 都抓)、`except Exception`: (不抓退出类)、`except (X, Y)`: (只抓指定)。通用性越强，风险越大。
- 底层实现：`BaseException` 是异常体系的总根；`SystemExit`、`KeyboardInterrupt`、`GeneratorExit` 直接继承它，而普通错误（`ZeroDivisionError` 等）都继承 `Exception`。所以 `except Exception` 通过“子类即匹配”的规则天然放过了退出类。
- 设计原因：把退出与中断设计成异常、但又不挂在 `Exception` 名下，是刻意的分层——普通代码的“捕捉一切”不应误吞用户的 `Ctrl+C` (`KeyboardInterrupt`) 或 `sys.exit()` (`SystemExit`)，否则程序将无法被用户正常终止。
- 实际问题：空 `except` 的典型危害是吞 bug：代码里有个拼写错误，却被兜底处理器默默吞掉，线上毫无日志，直

到用户投诉。这也是为什么 Python 社区流传"空 except 是代码坏味道"的说法。

·初学者误区：写 `try: ... except: pass` 试图"压制错误"，结果连 `KeyboardInterrupt` 都被吞，程序按 `Ctrl+C` 都没反应；或者把 `except Exception` 当成"绝对安全"——它照样会掩盖逻辑 bug。生活类比：空 `except` 像"不管屋里什么声音都关掉警报"——小偷（真 bug）也一起被放过了。

代码分析

```
try:
    ...
except NameError:
    ...
except IndexError:
    ...
except:
    ...
else:
    ...
```


NameError
IndexError

SystemExit / KeyboardInterrupt

·解释器如何处理：匹配按顺序进行：`NameError` → `IndexError` → 空 `except`（无条件命中）。注意空 `except` 在字节码里对应一个不比较任何类型的处理槽，`SystemExit`（`sys.exit()` 引发）和 `KeyboardInterrupt`（`Ctrl+C` 引发）也会被它接住——这正是它"危险"的原因。

·设计思想：本书这里埋伏了一个教学伏笔：`else` 与空 `except` 外观相似（都以 `:` 结尾），但职责天差地别——`else` 只管"没异常"的分支，绝不拦截任何异常。把两者放在一起展示，就是要你一眼分清"拦截"与"成功"。

34.4.4 Catching the No-Exception Case with else (用 else 捕捉"无异常"的情况)

英文原文

All told, else can be used in three places in Python: in if selections, for and while loops, and try exception handlers. In the latter, else is run when no exception occurs—not for unmatched exceptions (that's what the prior section's empty except is for). This else role may seem different than in if and loops, but this context differs. The need for an else clause in try is not always obvious to Python beginners. Without it, though, there is no direct way to tell whether the flow of control has proceeded past a try statement because no exception was raised, or because an exception occurred and was handled. Either way, we wind up after the try:

```
python try: ... # 运行代码except IndexError: ... # 处理异常# 我们到这里，是因为 try 失败了还是没失败？ Of course, we could initialize, set, and check a Boolean flag to know what happened, which adds lines of admin code. Much like the way else clauses in loops make the exit cause more apparent (for exits sans break), the else clause provides syntax in try that makes the outcome unambiguous with minimal extra code: python try: ... # 运行代码except IndexError: ... # 处理异常else: ... # 没有异常发生 You can almost emulate an else clause
```

by moving its code into the try block: `python try: ... # 运行代码... # 没有异常发生except IndexError: ... # 处理异常` This can lead to incorrect exception classifications, though. If the "no exception occurred" action itself causes an `IndexError`, it will register as a failure of the try block and erroneously trigger the exception handler below the try (unlikely perhaps, but true). By using an explicit else clause instead, you make the logic more obvious and guarantee that except handlers will run only for real failures in the code you're wrapping in a try, not for failures in the else no-exception case's action.

中文翻译

总而言之，else 在 Python 中可以用在三个地方：if 选择中、for 和 while 循环中，以及 try 异常处理器中。在后一种情况下，else 在没有异常发生时运行——而不是为未匹配的异常运行（那是上一节空 except 的职责）。这个 else 角色看起来可能与 if 和循环中的不同，但这里的环境本来就不同。对 Python 初学者来说，try 中 else 子句的必要性并不总是显而易见。但没有它，就无法直接判断控制流越过 try 语句，是因为没有异常被引发，还是因为发生了异常且被处理了。两种情况我们都会来到 try 之后：`python try: ... # 运行代码except IndexError: ... # 处理异常# 我们到这里，是因为 try 失败了还是没失败？当然，我们可以初始化、设置并检查一个布尔标志来知道发生了什么——这就要多写几行管理性代码。正如循环中的 else 子句让退出原因更明显（针对没有 break 的退出），try 中的 else 子句`

提供了用最小额外代码就让结果毫无歧义的语法：`python try: ... # 运行代码except IndexError: ... # 处理异常else: ... # 没有异常发生`你几乎可以通过把 `else` 的代码挪进 `try` 块来模拟它：`python try: ... # 运行代码... # 没有异常发生except IndexError: ... # 处理异常`但这可能导致错误的异常归类。如果“没有异常发生”的动作本身引发了 `IndexError`，它会被登记为 `try` 块的失败，从而错误地触发 `try` 下方的异常处理器（也许不太可能，但确实如此）。改用显式的 `else` 子句，你就让逻辑更清晰，并保证 `except` 处理器只为 `try` 所包裹代码中的真正失败运行，而不会为 `else` “无异常”分支的动作里的失败运行。

深度理解

- 核心概念：`else` 的使命是消除歧义——“走到 `try` 之后”这件事本身说明不了任何事，`else` 明确告诉你“成功路径在此”。
- 底层实现：字节码层面，`else` 块与 `except` 块的跳转目标不同：异常处理器只在异常发生时进入；`else` 在 `try` 块正常结束（`ENDFINALLY` 正常路径）时进入，两者互斥且不可能同时执行。
- 设计原因：不用标志变量的原因是“管理性代码”（`admin code`）会增加出错面——每多一行状态代码就多一个出 `bug` 的机会。语法本身表达意图，比程序员自己维护标志更可靠。
- 实际问题：经典场景是“读文件 → 处理数据”：`except` 处理“读不到”的错误，`else` 里做只有“读到数据”才该做的事（如写日志“解析成功”）。如果把成功逻辑塞进 `try`，处

理逻辑自身抛错会误导你以为是读取失败。

·初学者误区：以为 `else` 会"在异常被处理之后"执行——恰恰相反，只要发生过异常，`else` 就绝不执行。另一个误区：把 `else` 当 `except` 用，以为它能捕捉异常——它连一个异常都不抓。

代码分析

```
try:
    ...
    ...
except IndexError:
    ...
# "" try →
```

·解释器如何处理：把成功动作放进 `try` 后，解释器无法区分异常来自"主体代码"还是"成功动作"——对解释器而言它们都在同一个受保护区域内。而 `else` 是独立于受保护区域的代码块，它的异常不会被下面的 `except` 拦截。

·设计思想：这正是"职责分离"（separation of concerns）在语法层面的体现——`try` 块只放"可能失败的原始操作"，成功后的衍生操作放 `else`，让错误的归属一目了然。

34.4.5 Example: Default Behavior (示例：默认行为)

英文原文

Because the control flow through a program may be easier to capture in Python than in English, let's run s

ome simple examples that further illustrate exception basics with real code in files.

As noted, exceptions not caught by try statements percolate up to the top level of the Python process and run Python's default exception-handling logic (i.e., Python terminates the running program and prints a standard error message). To illustrate, module file `crashware.py` coded in Example 34-1 generates a divide-by-zero exception—by design.

Example 34-1. `crashware.py`

```
python def gobad(x, y): return x / y
def gosouth(x):
    print(gobad(x, 0))
if name == 'main':
    gosouth(1)
Because the program ignores the exception it triggers, Python kills the program and prints a message (edited here to condense paths and drop some interfaces' code-pointer lines for space):
```

```
text $ python3 crashware.py
Traceback (most recent call last):
  File ".../LP6E/Chapter34/crashware.py", line 7, in <module>
```

```
if name == 'main':
    gosouth(1)
  File ".../LP6E/Chapter34/crashware.py", line 5, in gosouth
```

```
    print(gobad(x, 0))
  File ".../LP6E/Chapter34/crashware.py", line 2, in gobad
```

```
    return x / y
ZeroDivisionError: division by zero
```

This message consists of a stack trace ("Traceback") and the name of and details about the exception that

was raised. The stack trace lists all lines active when the exception occurred, from oldest to newest. Note that because this code was written in a file instead of a REPL, the file and line-number information is more useful here. For example, we can see that the bad divide happens at the last entry in the trace—line 2 of the file `crashware.py`, a return statement.

Because Python detects and reports all errors at runtime by raising exceptions like this, exceptions are intimately bound up with the ideas of error handling and debugging in general. If you've worked through this book's examples, you've undoubtedly seen an exception or two along the way—even typos usually generate a `SyntaxError` or other exception when a file is imported or executed (that's when the code compiler is run).

By default, programming errors generate a useful error display like the one just shown, which helps you track down the problem. In some interfaces, this message today even comes with pointers to offending expressions, as well as speculative but mandatory "Did you?" tips (whose merit you may wish to reweigh later in your Python career):

```
text $ python3 >> import crashware >> crashware.gobad(1, 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File ".../LP6E/Chapter34/crashware.py", line 2, in gobad
```

return x / y ~ ~ ^ ~ ~ ZeroDivisionError: division by zero

Often, this standard error message is all you need to resolve problems in your code. For more heavy-duty debugging jobs, you can catch exceptions with try statements, or use debugging tools introduced in Chapter 3, such as the pdb standard-library module. See "Debugging Python Code" for more related tips.

中文翻译

因为程序中的控制流用 Python 代码来捕捉比用英语描述更容易，让我们运行一些简单示例，用文件里的真实代码进一步说明异常基础知识。如前所述，未被 try 语句捕捉的异常会一路渗透（percolate）到 Python 进程的顶层，并运行 Python 的默认异常处理逻辑（也就是：Python 终止正在运行的程序，并打印一条标准错误信息）。为了说明这一点，示例 34-1 中的模块文件 crashware.py 故意制造了一个除零（divide-by-zero）异常。示例 34-1. crashware.py

```
python def gobad(x, y): return x / y
def gosouth(x): print(gobad(x, 0))
if name == 'main': gosouth(1)
```

因为这个程序忽略了自己触发的异常，Python 终止程序并打印一条信息（此处为节省篇幅压缩了路径、删去了一些接口的代码指针行）：

```
$ python3 crashware.py
Traceback (most recent call last):
  File ".../LP6E/Chapter34/crashware.py", line 7, in <module>
    if name == 'main': gosouth(1)
  File ".../LP6E/Chapter34/crashware.py", line 5, in gosouth
    print(gobad(x, 0))
  File ".../LP6E/Chapter34/crashware.py", line 2, in gobad
    return x / y
ZeroDivi
```


sionError: division by zero 这条信息由堆栈跟踪 (stack trace, "Traceback") 和所引发异常的名称及细节组成。堆栈跟踪按从旧到新的顺序列出了异常发生时所有活跃的代码行。注意：由于这段代码写在文件里而非 REPL 中，这里的文件和行号信息更有用。例如，我们可以看出糟糕的除法发生在跟踪的最后一条目——crashware.py 文件第 2 行，一条 return 语句。因为 Python 正是这样通过引发异常在运行时检测和报告所有错误的，异常与错误处理 (error handling) 和调试 (debugging) 的整体思想紧密相连。如果你跟着本书的示例做过练习，你在路上无疑已经见过一两个异常了——甚至拼写错误通常在文件被导入或执行时也会产生 SyntaxError 或其他异常（因为那时代码编译器 (compiler) 会被运行）。默认情况下，编程错误会产生像刚才展示的那样有用的错误显示，帮助你追踪问题。如今在某些接口中，这条信息甚至带有指向出错表达式的指针，以及推测性但强制的“你是不是？” (“Did you?”) 提示（这些提示的价值，你也许想在之后的 Python 职业生涯中重新掂量）：

```
text $ python3 >>> import crashware >>> crashware.gobad(1, 0) Traceback (most recent call last): File "<stdin>", line 1, in <module> File "/.../LP6E/Chapter 34/crashware.py", line 2, in gobad return x / y ~ ~ ^ ~ ~ ZeroDivisionError: division by zero
```

通常，这条标准错误信息就是你解决问题所需的全部。对于更重型的调试任务，你可以用 try 语句捕捉异常，或使用第 3 章介绍的调试工具，如标准库的 pdb 模块。更多相关技巧见“Debugging Python Code”（调试 Python 代码）。

深度理解

·核心概念：默认行为 = 不设任何 try 时，异常一路冒泡到顶层，Python 打印 traceback 并终止程序。异常与调试 (debugging) 是同生共死的一对：Python 靠异常报告一切运行时错误。

·底层实现：traceback 由解释器在异常传播时逐帧记录（每帧的代码对象、行号、指令指针）；sys.excepthook 最终把它们格式化为“从最外层到最内层”的列表——最早调用的帧在上，错误发生的帧在最后（最接近真正 bug）

·设计原因：让“未处理异常 = 终止程序 + 完整 traceback”，是因为脚本语言的价值之一就是快速定位：栈顶帧 + 行号 + 出错表达式指针（3.11+ 的 `~~^~~` 标记），让 bug 无处遁形。

·实际问题：学会读 traceback 是 Python 程序员的基本功——从下往上读（最后一行是异常类型和消息，往上是出错调用链）。REPL 里报错不会杀进程，脚本文件里则会，这种差异常让人困惑。

·初学者误区：看到一大片 traceback 就慌，只盯最后一行。其实中间每一行都是一次函数调用，真正的现场是最靠下的那帧（`return x / y`）。另外注意 REPL（`<stdin>`）与文件的区别：文件模式下行号才是可靠的地图。

代码分析

```
def gobad(x, y):
    return x / y                                # 2

def gosouth(x):
    print(gobad(x, 0))                          # 5

if __name__ == '__main__': gosouth(1)         # 7
```

·解释器如何处理：gosouth(1) 调用 gobad(1, 0)，执行 1 / 0 时 C 层运算发现除数为 0，抛出 ZeroDivisionError 实例。该异常沿调用栈逐层上抛：gobad 帧没有处理器 → gosouth 帧没有 → 模块顶层没有 → 到达进程顶层，sys.excepthook 打印"最旧到最新"的三层帧信息和异常细节，然后进程以非零状态退出。

·设计思想：作者故意让代码"三层嵌套"——异常必须能穿透任意深度的调用栈，这正是异常与 goto 跳转的本质区别：它是与调用深度无关的控制流转移。设计上有意为"哪里都可能出错"的现实服务。

34.4.6 Example: Catching Built-in Exceptions (示例：捕捉内置异常)

英文原文

Python's error checking and default exception handling is often exactly what you want: especially for code in a top-level script file, an error often should terminate your program immediately. For many programs, there is no need to be more specific about errors in y

our code.

Sometimes, though, you'll want to catch errors and recover from them instead. If you don't want your program terminated when Python raises an exception, simply catch it by wrapping the program logic in a try. This is an important capability for programs such as network servers, which must keep running persistently. For example, the code in Example 34-2, file `kaboom.py`, catches and recovers from the `TypeError` Python raises immediately when you try to concatenate a list and a string (remember, the `+` operator expects the same sequence type on both sides).

Example 34-2. `kaboom.py`

```
python def kaboom(x, y): print(x + y) # 触发 TypeError
def serve(n=2): # 模拟长时间运行的任务
    for i in range(n):
        try: kaboom([1, 2], 'hack')
```

```
except TypeError: # 在这里捕捉并恢复
    print('Hello world!')
    print('Resuming here...') # 无论有无异常都继续到这里
if name == 'main':
    serve()

When the exception occurs in the function kaboom, control jumps to the try statement's except clause, which prints a message. Since an exception is "dead" after it's been caught like this, the program continues executing below the try rather than being terminated by Python. In effect, the code processes and clears the error, and your script recovers:
```

text \$ python3 kaboom.py Hello world! Resuming her
e... Hello world! Resuming here...

Keep in mind that once you've caught an error, control resumes at the place where you caught it (i.e., after the try); there is no direct way to go back to the place where the exception occurred (here, in the function kaboom). This makes exceptions more like simple jumps than function calls—there is no way to "return" to the scene of the crime.

中文翻译

Python 的错误检查和默认异常处理常常正是你想要的：尤其对顶层脚本文件中的代码来说，错误应该立即终止你的程序。对许多程序而言，没有必要对代码中的错误做更精细的处理。不过，有些时候你会想捕捉错误并从错误中恢复。如果你不想让程序在 Python 引发异常时被终止，只需把程序逻辑包进 try 来捕捉它。这对网络服务器这类必须持续运行的程序来说是一项重要能力。例如，示例 34-2 中文件 kaboom.py 的代码，捕捉并恢复了当你尝试拼接列表 (list) 和字符串 (string) 时 Python 立即引发的 TypeError (记住，+ 运算符期望两侧是相同的序列类型)。示例 34-2. kaboom.py

```
python def kaboom(x, y): print(x + y) # 触发 TypeError
def serve(n=2): # 模拟长时间运行的任务
    for i in range(n):
        try: kaboom([1, 2], 'hack')
        except TypeError: # 在这里捕捉并恢复
            print('Hello world!')
            print('Resuming here...')
    # 无论有无异常都继续到这里
if name == 'main':
    serve()
# 当异常发生在函数 kaboom 中时，控制流跳到 try 语句的 except 子句，它打印一条消息。由于异常
```

被这样捕捉后就是"死的" (dead) 了, 程序会在 try 下方继续执行, 而不会被 Python 终止。实际上, 代码处理并清除了错误, 你的脚本得以恢复: `text $ python3 kaboom.py` Hello world! Resuming here... Hello world! Resuming here... 请记住: 一旦你捕捉了错误, 控制流就在捕捉点 (也就是 try 之后) 恢复; 没有任何直接方法回到异常发生的地方 (这里是函数 kaboom 内部)。这使得异常更像简单的跳转 (jump) 而非函数调用——没有办法"返回到犯罪现场"。

深度理解

- 核心概念: 捕捉异常 = 把"致命错误"降级为"可处理事件"。`kaboom` 展示了错误恢复 (recovery) 的核心模式: 异常被 `except` 处理后, 程序继续跑。
- 底层实现: `except TypeError` 匹配成功后, 解释器清除异常状态 (`PyErrClear`), 从 `try` 语句之后的字节码继续执行——这就是"异常已死"的字面含义: 它不再存在, 无法继续传播。
- 设计原因: 异常 $\neq$ 返回。作者强调"不能回到犯罪现场", 是因为异常只携带"向上跳"的信息, 不携带"向下回"的路径——这种单向性让错误处理不可能陷入"重放错误现场"的死循环。
- 实际问题: 服务器模型: `serve()` 的循环每次迭代都包 `try`, 单个请求的 `TypeError` 只打一句"Hello world!" (这里用打印模拟容错处理), 循环继续处理下一个请求——这正是 Web 框架里每个请求都被 `try` 包裹的原理。

·初学者误区：以为捕捉异常后能"从出错的那一行继续执行"——不能！执行点只能从 try 之后继续。还有初学者把 try 块内的代码写得巨大，指望"哪里出错都行"——捕捉范围越大，恢复策略就越难精准。

代码分析

```
def kaboom(x, y):
    print(x + y)                                # TypeError+

def serve(n=2):                                #
    for i in range(n):
        try:
            kaboom([1, 2], 'hack')            #
        except TypeError:                      #
            print('Hello world!')
            print('Resuming here...')          #

if __name__ == '__main__': serve()
```

·解释器如何处理：第一次循环：kaboom([1, 2], 'hack') 执行 [1, 2] + 'hack'，PyObject 层发现 list 与 str 不能相加，抛出 TypeError。该异常在 kaboom 帧内无匹配处理器，上抛到 serve 帧——这里 try 匹配 TypeError，命中！执行 print('Hello world!')，异常清除，继续执行 print('Resuming here...')。第二次循环重复同一剧本。+ 运算符的"同类型序列"约束在 C 层的 sequenceconcat 中检查。

·设计思想：注意 print('Resuming here...') 在 try/except 外面、循环里面——这正是"每轮迭代独立容错、整体持续运行"的服务器模式雏形：错误被拦截在单次请求内，服务本身不中断。

34.5 The finally Clause (finally 子句)

英文原文

The other main flavor of the try statement is for coding finalization (a.k.a. termination) actions and is really related to exceptions only incidentally. If a finally clause is included in a try, Python will always run its block of statements "on the way out" of the try statement—whether an exception occurred while the try block was running or not. That is, this clause doesn't catch exceptions, it works around them. When used in isolation, this flavor's general form is this:

```
python try: statements # 先运行这个动作 finally: statements # 退出时总是运行这段代码
```

When a finally appears in try, Python begins by running the statement block associated with the try header line as usual. What happens next depends on whether an exception occurs during the try block, and what other clauses are present: Exception and match — If an exception occurs during the try block's run and is matched by an except clause, Python first runs the matching except block and then runs the finally block. After both finish, the program then resumes below the entire try statement. The finally is also run if the except raises a new exception. Exception and no match — If an exception occurs during the try block's run b

ut is not caught by an `except`, Python still comes back and runs the `finally` block, but it then propagates the exception up to a previously entered `try` or the top-level default handler. That is, `finally` is run even if an exception is raised and uncaught, but unlike an `except`, the `finally` does not terminate the exception—it continues being raised after the `finally` block runs. No exception — If an exception does not occur while the `try` block is running, Python first runs the `else` block (if present) and then runs the `finally` block. After both finish, the program then resumes below the entire `try` statement. The `finally` is also run if the `else` raises a new exception. The `try/finally` form is useful when you want to be completely sure that an action will happen after some code runs, regardless of the exception behavior of the code. In practice, it allows you to specify cleanup actions that always must occur, such as file closes and server disconnects where required. Technically speaking, `finally` can appear in the same statement as `except` and `else`, so there is really a single `try` statement with many optional clauses. Because of its distinct role, though, as well as its ordering rules we will meet in a moment, the `finally` clause may be best thought of as a distinct tool. Whether mixed or not, `finally` serves the same purpose—to specify cleanup actions that must always be run, regardless of any exceptions. As you'll also see later in this chapter, the `with` statement and its context managers provide an obj

ect-based way to do similar work for exit actions. Unlike finally, this statement also supports entry actions, but it is limited in scope to objects that implement the context-manager protocol it employs.

中文翻译

try 语句的另一主要形态是用于编写终结化 (finalization, 又名终止 termination) 动作, 它与异常的关系其实只是顺带相关。如果 try 中包含 finally 子句, Python 在"离开" try 语句时总是运行它的语句块——无论 try 块运行期间是否发生异常。也就是说, 这个子句不捕捉异常, 而是绕开它们工作。单独使用时, 这一形态的通用形式如下: `python try: statements # 先运行这个动作 finally: statements # 退出时总是运行这段代码` 当 finally 出现在 try 中时, Python 照常先运行与 try 头部行关联的语句块。接下来发生什么, 取决于 try 块期间是否发生异常, 以及还有哪些其他子句在场: 异常且匹配——如果 try 块运行期间发生异常, 并被某个 except 子句匹配, Python 先运行匹配的 except 块, 再运行 finally 块。两者都完成后, 程序在整条 try 语句下方继续。即使 except 引发了新异常, finally 也照样运行。异常但不匹配——如果 try 块运行期间发生异常, 但没有被任何 except 捕捉, Python 仍然会回来运行 finally 块, 但之后会把异常传播到此前进入的某个 try 或顶层默认处理器。也就是说: 即使异常被引发且未被捕捉, finally 也会运行; 但与 except 不同, finally 不会终结异常——异常在 finally 块运行后继续被引发。无异常——如果 try 块运行期间没有发生异常, Python 先运行 else 块 (若有), 再运行 finally 块。两者都完成后, 程序在整条 try 语句

下方继续。即使 `else` 引发了新异常，`finally` 也照样运行。当你想要完全确定某个动作会在某段代码运行之后发生——无论这段代码的异常行为如何——`try/finally` 形式就很有用。在实践中，它让你指定必须始终发生的清理动作（cleanup action），比如按需关闭文件和断开服务器连接。严格来说，`finally` 可以与 `except` 和 `else` 出现在同一条语句中，所以实际上只有一条带多个可选子句的 `try` 语句。不过，鉴于它独特的作用，以及我们马上要讲到的顺序规则，把 `finally` 子句当作一件独立的工具来思考也许最好。无论混用与否，`finally` 服务于同一个目的——指定无论任何异常都必须运行的清理动作。你还将在本章后面看到，`with` 语句及其上下文管理器（context manager）为退出动作提供了基于对象的类似方式。与 `finally` 不同，该语句还支持进入（entry）动作，但它的适用范围限于实现了它所使用的上下文管理器协议的对象。

深度理解

·核心概念：`finally` 是“出逃通道上的守门员”——无论 `try` 块是正常退出、被 `except` 处理后退出、还是异常未捕捉直接弹出，`finally` 都必然执行。它不“处理”异常，只保证“收尾”。

·底层实现：字节码层面，`finally` 块会被编译为可被两种路径进入的代码：正常路径（`try` 结束后顺序执行）和异常路径（解释器展开栈时通过 `SETUPFINALLY` 处理器跳入）。`finally` 块执行完，异常若仍存在则继续传播（`RERAISE` 指令），这就是“不终结异常”的实现。

·设计原因：资源（文件、锁、数据库连接）的生命周期管

理是编程的永恒难题——忘记释放 = 资源泄漏。finally 把"必然清理"从程序员的自觉变成语言的保证。

·实际问题：任何"打开就必须关闭"的资源都该配 finally：文件、socket、数据库游标、GUI 窗口、进程锁。这也是 with 语句存在的动机——它在对象层面自动化了这套模式。

·初学者误区：以为"有 finally 就能吞掉异常"——不能！finally 不匹配、不拦截、不终结异常，它只是"顺便打扫战场"。另一个经典误区：在 finally 里 return 会覆盖 try 中的返回值（高级陷阱，但本书此处不展开）。

代码分析

```
try:
    statements
finally:
    statements
```

·解释器如何处理：解释器把 finally 块安装为"两条路的汇合点"：若 try 正常完成，直接落入 finally；若中途异常，先保存异常状态，执行 finally 块，再把保存的异常恢复并继续向上传播（若存在匹配的外层 except 则交给它）。这保证了三条路径（匹配/不匹配/无异常）都经过 finally。

·设计思想："围绕（work around）异常"这个说法很精辟：finally 不在"异常是否发生"的决策线上，而在"退出路径"上——它的语义是"离开必经过"，与异常种类完全解耦。

34.5.1 Example: Coding Termination Actions with try/finally (示例：用 try/finally 编写终止动作)

英文原文

We coded some simple try/finally examples in the prior chapter. Example 34-3 lists a more tangible example that illustrates a typical role for this statement.

Example 34-3. closer.py

```
python class MyError(Exception): pass def stuff(file): f
ile.write('Hello?') # 可能滞留在文件缓冲区
```

```
raise MyError() # <= 用 # 启用或禁用我if name == 'mai
n': file = open('temp.txt','w') # 打开输出文件（这步也可
能失败） try: stuff(file) # 引发异常 finally: file.close() #
总是关闭文件以刷新输出缓冲区
```

```
print('Am I reached?') # 仅当无异常时才继续到这里 Wh
en the function in this code raises its exception, the c
ontrol flow jumps back and runs the finally block to c
lose the file. The exception is then propagated on to
either another try or the default top-level handler, w
hich prints the standard error message and shuts do
wn the program. Hence, the statement after this try is
never reached:
```

```
text $ python3 closer.py Traceback (most recent call l
ast): File "/.../LP6E/Chapter34/closer.py", line 10, in <
```

```
module> stuff(file) # 引发异常 File ".../LP6E/Chapter3  
4/closer.py", line 5, in stuff
```

```
raise MyError() # <= 用 # 启用或禁用我MyError
```

If the function here did not raise an exception (e.g., by disabling its raise line with an added #), the program would still execute the finally block to close the file, but it would then continue below the entire try statement:

```
text $ python3 closer.py Am I reached?
```

In this specific case, we've wrapped a call to a file-processing function in a try with a finally clause to make sure that the file is always closed, and thus finalized, whether the function triggers an exception or not. This way, later code can be sure that the file's output buffer's content has been flushed from memory to disk. A similar code structure can guarantee that server connections are closed, GUI windows are closed, and so on.

As we learned in Chapter 9, file objects are automatically closed on garbage collection in standard Python (CPython); this is especially useful for temporary files that we don't assign to variables. However, it's not always easy to predict when garbage collection will occur, especially in larger programs or alternative Python implementations with differing garbage collection policies. The try statement makes file closes more explicit and predictable: it ensures that the file will be cl

used on block exit, regardless of whether an exception occurs or not.

This particular example's function isn't all that useful (it always raises an exception!), but wrapping calls in try/finally statements is a good way to ensure that your closing-time termination activities always run. All bets are off if Python itself crashes completely, of course, but this is exceedingly rare; because it detects errors as a program runs, hard crashes are usually caused by linked-in C extension code, outside of Python's scope.

As a preview, notice how the user-defined exception in Example 34-3 is defined with a class; as you'll learn more formally in the next chapter, exceptions must all be class instances, for reasonably good causes.

中文翻译

我们在前一章已经编写过一些简单的 try/finally 示例。示例 34-3 列出了一个更实在的例子，说明该语句的典型角色。

```
示例 34-3. closer.py
python class MyError(Exception):
    pass
def stuff(file):
    file.write('Hello?') # 可能滞留在文件缓冲区
    raise MyError() # <= 用 # 启用或禁用我
if name == 'main':
    file = open('temp.txt', 'w') # 打开输出文件（这步也可能失败）
    try:
        stuff(file) # 引发异常
    finally:
        file.close() # 总是关闭文件以刷新输出缓冲区
    print('Am I reached?') # 仅当无异常时才继续到这里
当代码中的函数引发它的异常时，控制流跳回来运行 finally 块以关闭文件。异常随后被传播到另一个 try 或顶层默认处理器，后者打印标准
```

错误信息并关闭程序。因此，这个 try 之后的语句永远不会执行到：text \$ python3 closer.py Traceback (most recent call last): File "/.../LP6E/Chapter34/closer.py", line 10, in <module> stuff(file) # 引发异常File "/.../LP6E/Chapter34/closer.py", line 5, in stuff raise MyError() # <= 用 # 启用或禁用我MyError 如果这里的函数没有引发异常（例如，通过加一个 # 禁用它的 raise 行），程序仍会执行 finally 块来关闭文件，但随后会在整条 try 语句下方继续：text \$ python3 closer.py Am I reached? 在这个具体案例中，我们把对文件处理函数的调用包裹在带 finally 子句的 try 中，以确保文件无论函数是否触发异常总是被关闭、被终结化。这样，后面的代码就可以确信文件输出缓冲区的内容已经从内存刷新（flush）到磁盘。类似的结构可以保证服务器连接被关闭、GUI 窗口被关闭，等等。正如我们在第 9 章学到的，在标准 Python (CPython) 中，文件对象在垃圾回收（garbage collection）时会自动关闭；这对我们不赋给变量的临时文件尤其有用。然而，垃圾回收何时发生并不总是容易预测，尤其在更大的程序中，或在使用不同垃圾回收策略的其他 Python 实现里。try 语句让文件关闭更显式、更可预测：它确保文件在块退出时被关闭，无论是否发生异常。这个例子中的函数其实没什么用（它总是引发异常！），但把调用包裹在 try/finally 语句中是确保“收尾时终止活动总是运行”的好方法。当然，如果 Python 自身彻底崩溃，一切承诺都作废——但这极其罕见；因为 Python 会在程序运行时检测错误，硬崩溃通常由链接进来的 C 扩展代码造成，超出了 Python 的管辖范围。作为预览，请注意示例 34-3 中的用户自定义异常是用 class 定义的；正如你将在下一章更正式学到的，异常必须是类的

实例，而且理由相当充分。

深度理解

·核心概念：try/finally 的经典资源管理模式——"打开 → 处理 → 必然关闭"。异常发生时清理动作照样执行，这就是"终止动作 (termination actions)"的含义。

·底层实现：file.close() 会把文件对象中缓冲区 (buffer) 里滞留的数据写回磁盘。write() 只是写入内存缓冲区，close() 触发 flush——这就是为什么"不关闭文件 = 丢数据"。

·设计原因：CPython 的引用计数保证文件对象在失去引用时立即关闭，但这是实现细节而非语言承诺 (Jython、PyPy 等可能延迟)；try/finally 把"何时关闭"从"垃圾回收的运气"变成"代码的确定性"。作者强调这点，是为了说明：不要依赖隐式行为。

·实际问题：这是 with 语句的直接前身——每写一次 try/finally 的打开-关闭都要 4 行样板代码，所以后来 Python 2.5 引入了 with 来压缩它。本章后半部分会正式对比两者。

·初学者误区：以为"反正垃圾回收会关文件"，于是在循环里大量打开文件却不关闭——缓冲区数据可能永远不落盘，程序也可能耗尽文件描述符。另一个误区：以为 finally 会阻止异常传播——closer.py 的输出显示异常照常打印 traceback。

代码分析

```
class MyError(Exception): pass                # Except...

def stuff(file):
    file.write('Hello?')                      #
    raise MyError()                          # <= #

if __name__ == '__main__':
    file = open('temp.txt', 'w')              #
    try:
        stuff(file)                          #
    finally:
        file.close()                        #
    print('Am I reached?')                   #
```

·解释器如何处理：执行 `stuff(file)` 时先 `write` 再 `raise MyError()`。异常弹出时解释器先执行 `finally` 块：`file.close()` 把缓冲的 'Hello?' 落盘。之后异常继续传播到顶层，打印 `MyError` 的 `traceback` 并终止程序，所以 `print('Am I reached?')` 永远不执行。若注释掉 `raise` 行：`try` 正常结束，`finally` 仍执行（关闭文件），然后打印 `Am I reached?`。

·设计思想：`close()` 语义 = "终结化 (finalize)"——它不只释放文件描述符，还保证缓冲数据完整交付。这个例子点出关键教训：清理动作的价值在于"无论成败都执行"，因为成败本身不是我们能提前预知的。同时注意 `raise MyError()` 中 `MyError(Exception)` 的定义——异常是类，这是第 35 章的主线。

34.6 Combined try Clauses (组合 try 子句)

英文原文

For the first 15 years of Python's tenure (more or less), the try statement came in two flavors and was two separate statements—we could either use a finally to ensure that cleanup code was always run, or write except blocks to catch and recover from specific exceptions and optionally specify an else clause for when exceptions occurred.

That is, the finally clause could not be mixed with except and else. This was partly because of implementation issues, and partly because the meaning of mixing the two seemed obscure—catching and recovering from exceptions seemed a disjoint concept from performing cleanup actions.

For better or worse, the two statements eventually merged. Today, we can mix finally, except, and else clauses in the same statement—in part because of similar utility in the Java language (alas, many a programming-language mod owes to imitation). That is, the try statement in its most complete form looks like this:

```
python try: # 组合 try 语句main-action
except Exception1: # 捕捉特定异常handler1
```

```
except Exception2: handler2
```

```
except: # 捕捉所有（其他）异常handler3
```

```
else: # 无异常处理器handler4 finally: # finally 包裹一切  
finally-block
```

The code in this statement's main-action block is executed first, as usual. If that code raises an exception, all the except blocks are tested, one after another, for a match to the exception raised: handler1 is run for Exception1, handler2 for Exception2, and handler3 for all others. If no exception is raised, handler4 is run.

No matter what's happened previously, the finally-block is executed once, after the main action block is exited and any handler block has been run. In fact, the code in the finally-block will be run even if an error or raise in an except or else block causes a new exception to be raised.

As outlined earlier, even in mixed usage like this, the finally clause does not end the exception—if an exception is active when the finally-block is executed, it continues to be propagated after the finally-block runs, and control jumps somewhere else in the program (to an earlier try, or to the default top-level handler). If no exception is active when the finally is run, control resumes after the entire try statement.

The net effect is that the finally is always run, regardless of whether: - An exception occurred in the main a

ction and was handled. - An exception occurred in the main action and was not handled. - No exceptions occurred in the main action. - A new exception was triggered in one of the handlers.

Again, the finally serves to specify cleanup actions that must always occur on the way out of the try, regardless of what exceptions have been raised or handled.

中文翻译

在 Python 历史的前 15 年（大致如此），try 语句有两种形态、是两条独立的语句——我们要么用 finally 确保清理代码总是运行，要么写 except 块来捕捉特定异常并恢复、可选地为“发生异常时”指定一个 else 子句。也就是说，finally 子句不能与 except 和 else 混用。这部分是因为实现上的问题，部分是因为两者混用的含义似乎很晦涩——“捕捉并恢复异常”与“执行清理动作”看起来是两个不相干的概念。不管好坏，这两条语句最终还是合并了。今天，我们可以在同一条语句中混用 finally、except 和 else 子句——部分原因是 Java 语言中有类似的实用设计（唉，编程语言的许多变革都要归功于模仿）。也就是说，try 语句最完整的形式长这样：

```
python try: # 组合 try 语句
main-action
except Exception1: # 捕捉特定异常
handler1
except Exception2: handler2
except: # 捕捉所有（其他）异常
handler3
else: # 无异常处理器
handler4
finally: # finally 包裹一切
finally-block
```

本语句 main-action 块中的代码照常先执行。如果该代码引发异常，所有 except 块会一个接一个地测试是否与引发的异常匹配：Exception1 运行 handler1

er1, Exception2 运行 handler2, 其他一切运行 handler 3。如果没有引发异常, 则运行 handler4。无论之前发生了什么, finally-block 都会执行一次——在退出主动作块、且任何处理器块已运行之后。事实上, 即使 except 或 else 块中的错误或 raise 引发了新异常, finally-block 中的代码也照样运行。如前所述, 即使这样混用, finally 子句也不会终结异常——如果执行 finally-block 时异常仍处于活跃状态, 它在 finally-block 运行后会继续被传播, 控制流跳到程序中的别处(某个更早的 try, 或默认顶层处理器)。如果执行 finally 时没有活跃异常, 控制流在整条 try 语句之后继续。净效果是: 无论以下哪种情况, finally 都总是运行: - 主动作中发生异常且已被处理。- 主动作中发生异常且未被处理。- 主动作中没有任何异常发生。- 某个处理器中触发了新异常。再说一次, finally 服务于"指定在离开 try 时必须发生的清理动作", 无论引发或处理了哪些异常。

深度理解

·核心概念: 组合 try = 一张"完整保险单": except 是风险种理赔、else 是"没出事"奖励、finally 是"无论是否理赔都要走完的流程"。四种场景下 finally 都执行。

·底层实现: CPython 对组合形式生成的字节码实际上等价于"try/finally 里嵌套 try/except/else"——finally 被实现为最外层的处理器, 任何内部路径(包括 except 里再抛异常)都必须穿过它。

·设计原因: 历史合并动机: Java 的 try-catch-finally 已经证明了组合的实用性; 同时实现障碍(旧编译器难处理

混合) 最终被克服。作者调侃"编程语言的变革多源于模仿"。

·实际问题：最完整的错误处理结构——"分类型处理错误、成功分支做事后动作、无论如何都释放资源"——在一个语句里表达，不再需要嵌套两层 try。

·初学者误区：以为 except 处理完异常后，finally 之前还有"继续往下"的机会——实际上 finally 是最后一站。还有人不理解"finally 不终结异常"：即使有 finally，未处理的异常照样杀死程序。

代码分析

```
try:                                # try
    main-action                      #
except Exception1:                  #
    handler1
except Exception2:
    handler2
except:                              #
    handler3
else:                               #
    handler4
finally:                             # finally
    finally-block
```

·解释器如何处理：执行顺序为：主动作 → 若有异常按 Exception1 → Exception2 → 兜底顺序匹配并执行对应处理器（else 跳过）；若无异常执行 handler4。无论哪条路径，最后必然执行 finally-block。若某处理器自身抛出新异常，finally 照常执行，新异常随后向外传播。读者可自测四种情况：异常被处理、异常未处理、无异常、处理

器抛新异常——finally 均执行。

·设计思想：这张结构图就是全书最完整的异常语法速查表。注意 except: 空兜底与 else: 相邻出现——前者是"最后一个异常选项"，后者是"唯一一个无异常选项"，一字之差，天壤之别。

34.6.1 Combined-Clause Syntax Rules (组合子句的语法规则)

英文原文

When combined like this, the try statement must have either an except or a finally, and the order of its parts must be like this (where "->" means "is followed by"): try -> except -> else -> finally In this, the else and finally are optional, and there may be zero or more excepts, but there must be at least one except if an else appears. Really, the try statement consists of two parts: excepts with an optional else, and/or the finally .

In fact, it's more accurate to describe the combined try statement's syntactic by the following two alternative formats (where square brackets mean optional and star means any number of what precedes it):

python try: # 格式 1 statements

except [type [as value]]: statements [except [type [as value]]: statements] [else: statements] [finally: statem

ents]

try: # 格式 2 statements finally: statements

Because of these rules, the else can appear only if there is at least one except, and it's always possible to mix except and finally, regardless of whether an else appears or not. It's also possible to mix finally and else, but only if an except appears too (though the except can omit an exception name to catch everything and run a raise statement, described later, to reraise the current exception). If you violate any of these (arguably intricate!) ordering rules, Python will raise a syntax error exception before your code runs.

中文翻译

像这样组合时，try 语句必须包含 except 或 finally 之一，而且各部分的顺序必须如此（其中“->”表示“后面跟着”）：
try -> except -> else -> finally 其中 else 和 finally 都是可选的；except 可以有零个或多个；但只要出现 else，就必须至少有一个 except。实际上，try 语句由两部分组成：带可选 else 的若干 except，和/或 finally。事实上，用下面两种替代格式来描述组合 try 语句的语法更准确（方括号表示可选，星号表示前面内容可出现任意多次）：
python try: # 格式 1 statements except [type [as value]]: statements [except [type [as value]]: statements] [else: statements] [finally: statements] try: # 格式 2 statements finally: statements 由于这些规则，else 只有在至少有一个 except 时才能出现；而且无论是否出现 else，except 与 finally 总是可以混用。finally 与 else 也可以混用，但

前提是必须同时有一个 `except`（不过这个 `except` 可以省略异常名以捕捉一切，并运行一条稍后介绍的 `raise` 语句来重新引发当前异常）。如果你违反任何一条这些（可以说相当精细的！）顺序规则，Python 会在你的代码运行之前引发一个语法错误异常（`syntax error exception`）。

深度理解

·核心概念：语法规则一句话：`try -> except -> else? -> finally?`，且“`else` 需要至少一个 `except`；`try` 至少要有 `except` 或 `finally` 之一”。这是异常语法的“文法宪法”。

·底层实现：这些规则在编译期执行——Python 解析器（`parser`）在把源码编译成字节码时检查子句顺序，违规直接抛 `SyntaxError`，程序根本不会开始运行。

·设计原因：`else` 依赖 `except` 的语义理由是：`else` 是“`except` 体系的补充分支”——没有 `except`，“有异常 vs 无异常”的二分就不存在，`else` 也就失去参照。顺序固定则让解析器与读者都无需猜测。

·实际问题：写代码时最常见的语法错误就是顺序颠倒：`finally` 写在 `except` 前面、或 `else` 放在 `except` 前面。编译器会立刻报错，但报错位置有时令人困惑——记住这条顺序链就能少走弯路。

·初学者误区：以为“`try` 可以只有 `else` 没有 `except`”（想当然地觉得“无异常就干某事”）；或者写 `try: ... finally: ... except:`——顺序违规。这些都会在运行前被语法检查拦下。

代码分析

```
try:                                # 1 except + ...
    statements
except [type [as value]]:
    statements
[except [type [as value]]:
    statements
]*
[else:
    statements
]
[finally:
    statements
]

try:                                # 2 finally
    statements
finally:
    statements
```

·解释器如何处理：格式 1 与格式 2 是 try 的全部合法形态。格式 1 中 except 可重复零到多次、else 和 finally 各至多一次；格式 2 是"只要清理、不要捕捉"的极简形态。任何其他组合（如 try/else、try/finally/except）在解析阶段即被拒绝。

·设计思想：方括号与星号的文法表示法本身就是教学工具——它把"可选"与"可重复"可视化，比大段文字描述更精确。记住：格式 2 的存在说明"清理"是独立于"捕捉"的第一等需求。

34.6.2 Combining finally and Except by Nesting (通过嵌套组合 finally 与 except)

英文原文

It may help to realize that it's also possible to combine finally and except clauses in a try by syntactically nesting a try/except in the try block of a try/finally statement. We'll explore this technique more fully in Chapter 36, but the following has the same effect as the combined form shown at the start of this section:

```
python try: # 与组合形式等价的嵌套形式
    try: main-action
    except Exception1: handler1
    except Exception2: handler2
    except: handler3
else: handler4 finally: finally-block
```

Again, the finally block is always run on the way out, regardless of what happened in the main action and regardless of any exception handlers run in the nested try (trace through the four cases listed previously to see how this works the same). Since an else always requires an except, this nested form even sports the same mixing constraints of the combined form outlined in the preceding section.

However, this nested equivalent seems more obscure to some people and requires more code than the new merged form—though just one four-character line plus extra indentation. Mixing finally into the same statement might make your code easier to write and read, though this also might depend on who you ask.

中文翻译

认识到这一点也许有帮助：你也可以通过语法嵌套来组合 finally 和 except 子句——把一条 try/except 放在一条 try/finally 语句的 try 块里。我们将在第 36 章更全面地探索这项技术，不过下面的写法与本小节开头展示的组合形式效果相同：python try: # 与组合形式等价的嵌套形式
try: main-action
except Exception1: handler1
except Exception2: handler2
except: handler3
else: handler4
finally: finally-block
再说一次，finally 块在退出时总是运行——无论主动作发生了什么，也无论嵌套 try 中运行了哪些异常处理器（把前面列出的四种情况逐个推演一遍，就能看到运行方式完全一致）。由于 else 总是要求有 except，这个嵌套形式甚至与上一节所述组合形式有着相同的混用约束。不过，对有些人来说这个嵌套等价形式显得更晦涩，而且比新的合并形式需要更多代码——虽然只是多了一行四个字符的 try: 和额外的缩进。把 finally 混进同一条语句或许能让你的代码更易写易读，不过这也可能取决于你问谁。

深度理解

·核心概念：组合形式只是嵌套形式的语法糖——两者语义完全等价。这个等价关系是理解“组合 try”的钥匙。

- 底层实现：CPython 在解析组合形式时，内部正是按"嵌套"来生成代码对象的——finally 永远是外层保护，except 在里层。这解释了为什么 finally 能在 except 抛新异常时依然执行。
- 设计原因：先有嵌套，后有合并。作者展示嵌套形式，是为了让读者明白：合并形式不是"魔法"，而是"语法糖"——理解嵌套模型，就能推演组合形式的一切行为。
- 实际问题：调试时，如果你怀疑 finally 与 except 的交互诡异，可以把组合形式"拆开"成嵌套形式一步步看——两种写法运行时行为完全一样，拆开往往更清晰。
- 初学者误区：有人以为嵌套形式与组合形式"略有不同"（比如异常处理时机）——没有任何不同，它们是同一语义的两副面孔。区别只在代码风格与行数。

代码分析

```
try:                                # try/finally
    try:                            # try/except/else
        main-action                #
    except Exception1:
        handler1
    except Exception2:
        handler2
    except:
        handler3
    else:
        handler4
finally:
    finally-block                   #
```

- 解释器如何处理：异常发生时，先查最近进入的内层 try

的 except 链：命中则执行处理器并清除异常，内层正常结束；不命中则内层异常向外抛，但在离开内层之前，解释器会执行外层 try 的 finally——于是异常经过 finally 后继续向外传播。这正是“异常穿透嵌套帧时逐层展开”的生动演示。

·设计思想：这段代码是“异常传播的解剖图”：内层 try 决定“谁来处理”，外层 finally 决定“走前必须干什么”。二者职责不同、层级不同，组合形式的合并不过是一次文法的压缩。

34.6.3 Combined-Clauses Example (组合子句示例)

英文原文

To demo the effect of mixing finally with other try clauses, the script listed in Example 34-4, trycombos.py, codes four common scenarios, with print statements that describe the meaning of each.

Example 34-4. trycombos.py

```
python sep = '-' 45 + '\n' print(sep + 'EXCEPTION RAISE  
D AND CAUGHT') try: x = 'hack'[99] except IndexError:  
print('except run') finally:
```

```
print('finally run') print('after run') print(sep + 'EXCEPT  
ION NOT RAISED') try: x = 'hack'[3] except IndexError:  
print('except run') finally:
```

```
print('finally run') print('after run') print(sep + 'EXCEPT
ION NOT RAISED, WITH ELSE') try: x = 'hack'[3] except
IndexError: print('except run') else: print('else run') fin
ally:
```

```
print('finally run') print('after run') print(sep + 'EXCEPT
ION RAISED BUT NOT CAUGHT') try: x = 1 / 0 except
IndexError: print('except run') finally:
```

print('finally run') print('after run') When this code is run, the following output is produced. Trace through the code to see how exception handling produces the output of each of the four tests here:

```
text $ python3 trycombos.py-----
----- EXCEPTION RAISED AND CAUGHT

except run finally run after run -----
----- EXCEPTION NOT RAISED finally r
un after run -----
EXCEPTION NOT RAISED, WITH ELSE else run finally r
un after run -----
EXCEPTION RAISED BUT NOT CAUGHT finally run Tra
ceback (most recent call last): File "/.../LP6E/Chapter3
4/trycombos.py", line 38, in <module>

x = 1 / 0 ZeroDivisionError: division by zero
```

This example uses built-in operations in the main action to trigger exceptions (or not), and it relies on the fact that Python always checks for errors as code is running. The next section shows how to raise exceptio

ns manually instead.

中文翻译

为了演示把 finally 与其它 try 子句混用的效果，示例 34-4 中的脚本 trycombos.py 编码了四种常见场景，并用 print 语句说明每种场景的含义。示例 34-4. trycombos.py

```
python sep = '-' 45 + '\n'print(sep + 'EXCEPTION RAISED AND CAUGHT') # 异常被引发并被捕捉try: x = 'hack'[99] # 下标越界 → IndexError except IndexError: print('except run') finally: print('finally run') print('after run') print(sep + 'EXCEPTION NOT RAISED') # 未引发异常try: x = 'hack'[3] except IndexError: print('except run') finally: print('finally run') print('after run') print(sep + 'EXCEPTION NOT RAISED, WITH ELSE') # 未引发异常，带 else try: x = 'hack'[3] except IndexError: print('except run') else: print('else run') finally: print('finally run') print('after run') print(sep + 'EXCEPTION RAISED BUT NOT CAUGHT') # 异常被引发但未被捕捉try: x = 1 / 0 # Zero DivisionError, 不匹配 IndexError except IndexError: print('except run') finally: print('finally run') print('after run')
```

运行这段代码时，产生如下输出。请顺着代码推演，看异常处理如何产生这里四个测试各自的输出：text \$ python3 trycombos.py

```
-----EXCEPTION RAISED AND CAUGHT except run finally run after run-----
-----EXCEPTION NOT RAISED finally run after run-----
-----EXCEPTION NOT RAISED, WITH ELSE else run finally run after run--
```

-----EXCEPTION RAISED BUT NOT CAUGHT finally run Traceback (most recent call last): File ".../LP6E/Chapter34/trycombos.py", line 38, in <module> x = 1 / 0 ZeroDivisionError: division by zero 这个示例在主动作中使用内置运算来触发（或不触发）异常，它依赖的事实是：Python 总会在代码运行时检查错误。下一节将展示如何改为手动引发异常。

深度理解

·核心概念：四个实验 = 四种运行模式的"打印实验"：①异常被捕捉 → except + finally 都运行；②无异常 → 只有 finally；③无异常带 else → else + finally；④异常未捕捉 → 只有 finally，然后程序崩溃。四种场景 finally 全勤。

·底层实现：'hack'[99] 在解释器求值下标时做边界检查，越界抛 IndexError；1 / 0 在 C 层除法例程中抛 ZeroDivisionError——这些都是"运行时自动错误检查"的实例，无需程序员手动 raise。

·设计原因：用输出而非文字讲语义，是教学上的高招——读者亲手"推演 + 验证"一遍，比背规则记十年都牢。四条输出构成了 try 子句行为的完整测试矩阵。

·实际问题：这套组合是生产代码里最常用的错误处理骨架：try（危险操作）→ except（针对性恢复）→ else（成功善后）→ finally（资源清理）。可以把第④种场景理解为"兜底恢复失败"时程序的最终命运。

·初学者误区：从第①场景误以为"except 之后程序就结束

"——不是，after run 照常打印，异常已死。从第④场景误以为"finally 会打印后程序继续"——不，finally 打印完，ZeroDivisionError 继续传播，程序被终止，after run 没有机会打印。

代码分析

```
sep = '-' * 45 + '\n' # 45

# 1
try:
    x = 'hack'[99] # 'hack' 4
except IndexError:
    print('except run') # IndexError →
finally:
    print('finally run') #
print('after run') #

# 4
try:
    x = 1 / 0 # ZeroDivisionError #...
except IndexError:
    print('except run') #
finally:
    print('finally run') #
print('after run') #
```

·解释器如何处理：场景 1：字符串下标 99 触发 IndexError → 匹配 except IndexError → 打印 except run → 异常清除 → 执行 finally 打印 finally run → 继续打印 after run。场景 4：ZeroDivisionError 与 IndexError 无子类关系，不匹配 → 仍执行 finally 打印 finally run → 异常继续传播到顶层 → 打印 traceback、进程终止 → after run 永不可达。这正是"finally 不终结异常"的最直观演示

·设计思想：四个场景按"异常强度"递进排列（被接住 → 没发生 → 没发生+else → 没人接），每个场景只改一个变量，让读者像做对照实验一样精确归因：except 管"接住"，else 管"平安"，finally 管"收尾"——各司其职，互不越界。

34.7 The raise Statement (raise 语句)

英文原文

To trigger exceptions explicitly, code raise statements. Their general form is simple—a raise statement consists of the word `raise`, optionally followed by the class to be raised, or an instance of it:

```
python raise instance # Raise an instance of a class
raise class # Make and raise an instance of a class
raise # Reraise the most recent exception
```

As mentioned earlier, exceptions are always instances of classes today. Hence, the first `raise` form here is the most common—we provide an instance directly, either created before the `raise` or within the `raise` statement itself. If we pass a class instead, Python calls the class with no constructor arguments, to create an instance to be raised; this form is equivalent to adding parentheses after the class reference. The last form reraises the most recently raised exception; it's commonly used in exception handlers to propagate exceptions that have been caught. NOT

E Blast from the past: Long ago and far away (well, before Python 2.6), exceptions could be identified as simple string objects, with an optional associated data item in `raise`. This was replaced with classes to support added functionality and categories, as you'll see in the next chapter. Still, strings were a simpler model for simpler roles, didn't require newcomers to learn classes and OOP before exceptions, and didn't force some Python books to put exceptions on hold until Part VII!

中文翻译

要显式地触发异常，就编写 `raise` 语句。它们的一般形式很简单——一条 `raise` 语句由关键字 `raise` 组成，后面可选地跟一个要引发的类（class），或该类的一个实例（instance）：
`python raise instance # 引发一个类的实例`
`raise class # 创建并引发一个类的实例`
`raise # 重新引发最近一次的异常`
如前所述，如今的异常总是类的实例。因此，这里的第一种 `raise` 形式最常见——我们直接提供一个 instance，它要么在 `raise` 之前创建，要么就在 `raise` 语句自身内部创建。如果我们改而传入一个 class，Python 会用零个构造函数参数去调用这个类，以创建一个待引发的实例；这种形式等价于在该类引用后面加一对括号。最后一种形式会重新引发最近引发的异常；它常被用在异常处理器中，用来传播那些已被捕捉到的异常。注（NOTE）来自过去的回响（Blast from the past）：很久很久以前（准确说，在 Python 2.6 之前），异常可以只是简单的 string（字符串）对象，配合可在 `raise` 中提供的可选关联数据项。后来它被 cl

ass（类）取代，以支持更多的功能和分类——下一章你会看到这一点。不过，字符串对于简单的角色确实是一种更简单的模型：它不需要新手在接触异常之前就去学习类和 OOP，也不会迫使一些 Python 书籍把异常推迟到第七部分才讲！

深度理解

·核心概念：raise 是异常体系的"主动发起端"——之前所有示例里的异常都是 Python 在出错时自动抛出的；有了 raise，程序员就能手动制造异常。三种形式 = 给实例 / 给类 / 什么都不给。裸 raise (re-raise) 只允许在"有正在传播的异常"的上下文中使用。

·底层实现：raise class 与 raise class() 在字节码层面几乎等价——解释器都会调用该异常类的构造器创建一个实例再抛出。raise 裸形式是一条独立的 RAISEVARARGS 0 指令，它从线程状态对象里取出当前活跃异常并重新抛出。

·设计原因：允许"传类"是懒惰即美德 (laziness) 的体现——既然传实例时类会被自动调用，为何不让解释器替你调用？而保留裸 raise，是为了给"捕捉后想继续传播"的处理器一个简洁出口，避免写 raise sys.excinfo()[1] 这样的陈词滥调。

·实际问题：生产代码里最常见的 raise 是你自己抛业务异常（参数非法、逻辑不满足）或封装第三方库错误；裸 raise 则常见于日志记录器里——"记个日志，但错误还是要往上抛"。

·初学者误区：①把三种形式混为一谈，在 `except` 块外写裸 `raise`（报 `RuntimeError: No active exception to re-raise`）；②以为“传类”和“传实例”有细微区别——它们几乎完全等价；③怀念 Python 2 时代的字符串异常——那已成为历史，写 `raise "error"` 会直接报 `TypeError`。

代码分析

```
raise instance      #  
raise class         # Python  
raise               #
```

·解释器如何处理：第一条执行 `RAISEVARARGS` 指令时，解释器拿到的是现成的异常实例，直接进入“传播”流程；第二条会先对类执行一次无参构造调用（走 `new + init`），得到一个实例后再传播；第三条从当前线程的异常状态中取出活跃异常实例，原样重新抛出。三条路径最终都会在字节码里落成“设置异常三件套（类型、值、`traceback`）→ 跳出当前帧找处理器”的动作。

·设计思想：三种形式覆盖“你有实例、你有类、‘我什么都不管就是要重抛’”三种心智场景，`raise` 因此成为一条几乎零学习成本、却能精确表达意图的语句。

34.7.1 Raising Exceptions（引发异常）

英文原文

To make `raise` more concrete, let's turn to some examples. With built-in exceptions, the following two for

ms are equivalent—both raise an instance of the exception class named, but the first creates the instance implicitly: `python raise IndexError` # Class (instance created) `raise IndexError()` # Instance (created in statement) We can also create the instance ahead of time—because the raise statement accepts any kind of object reference, the following two examples raise `IndexError` just like the prior two: `python exc = IndexError()` # Create instance ahead of time `raise exc` `excs = [IndexError, TypeError]` `raise excs[0]` In fact, the instance provided to raise can also be obtained in the try that catches it, per the next section.

中文翻译

为了让 `raise` 更具体，让我们回头看看一些例子。以内置异常为例，下面两种形式是等价的——它们都引发所指异常类的一个实例，但第一种形式是隐式创建实例：`python raise IndexError` # 类形式（隐式创建实例）`raise IndexError()` # 实例形式（在语句中创建实例）我们还可以提前创建好实例——因为 `raise` 语句接受任何对象引用，下面两个示例与前两个一样引发 `IndexError`：`python exc = IndexError()` # 提前创建实例 `raise exc` `excs = [IndexError, TypeError]` `raise excs[0]` 事实上，提供给 `raise` 的实例甚至可以在捕捉它的那条 `try` 里被拿到——这是下一小节的主题。

深度理解

·核心概念：`raise` 的本质是“抛出一个对象”，该对象必须是异常类的一个实例。传给 `raise` 的“任何对象引用”意味

着你可以把异常实例存储在变量、列表、字典里再抛——异常实例就是普通对象。

·底层实现：`raise excs[0]` 会先执行下标操作取出列表元素，再以该对象进入异常机制——异常机制本身不关心这个对象是“刚构造的还是从篮子里翻出来的”。唯一约束是运行时校验：被抛对象必须是 `BaseException` 的实例（否则报 `TypeError: exceptions must derive from BaseException`）。

·设计原因：允许“先构造、后抛出”给了程序员组装复杂异常的余地：你可以给异常塞数据、绑定属性，甚至挑选“抛哪个”（如上例的 `excs[0]`），然后再交给 `raise`。

·实际问题：真实代码里最常见的写法其实是 `raise IndexError()`（一次成型）；“提前构造”多用于多分支里决定抛同一类异常但携带不同参数、或需要做日志/加工之后再抛的场合。

·新手常见错误：以为 `raise` 只能紧跟字面异常类名；以为 `raise<类名>` 与 `raise <类名>()` 行为有差（如构造参数）——本质完全一致，只是后者可以显式传构造参数。

代码分析

```
raise IndexError          # IndexError()
raise IndexError()        # raise

exc = IndexError()        #
raise exc                 # ""
excs = [IndexError, TypeError] #
raise excs[0]             # — raise
```

·解释器如何执行：执行 `raise excs[0]` 的字节码序列是：加载列表 → 下标 0 → 得到类对象 `IndexError` → 因为是"类"而非"实例"，解释器自动无参构造一个 `IndexError` 实例 → 进入异常传播流程（沿调用栈找处理器）。而 `raise exc` 拿到的是已构造的实例，跳过构造步骤直接传播。

·设计思想：异常与普通对象无差别待遇——这正是 Python"万物皆对象"哲学的又一例证。`raise` 只关心"你给的东西是不是合格异常"，至于它是字面量、变量还是列表元素，语法上完全自由。

34.7.2 The `except as` hook (except as 钩子)

英文原文

When an exception is raised, Python sends the raised instance along with the exception. If a try includes an `except` with an `as` clause per Table 34-1, the variable it gives will be assigned the instance raised by `raise` or Python: `python try: ... except IndexError as X: # X assigned the raised instance object ...` The `as` is optional in a try handler (if it's omitted, the instance is simply not assigned to a name), but including it allows the handler to access both data in the instance and methods in the exception class. This model works the same for user-defined exceptions we code with classes—the following, for example, passes to the exception clas

s constructor arguments that become available in the handler through the assigned instance: python class MyExc(Exception): pass try: raise MyExc('oops') # Exception class with constructor args except MyExc as X: # Instance attributes available in handler print(X.args) # Prints ('oops',) Because this almost encroaches on the next chapter's topic, though, we'll defer further details until then. Regardless of their source, exceptions are always identified by class instance objects, and at most one is active at any given time (sans the except groups of Chapter 35). Once caught by an except clause anywhere in the program, an exception ends and won't propagate to another try, unless it's reraised by another raise statement or error.

中文翻译

当异常被引发时，Python 会把被引发的实例（instance）随异常一起送去。如果某条 try 的 except 带 as 子句（见 Table 34-1），则该子句给出的变量会被赋值为由 raise 或 Python 引发的那个实例对象：python try: ... except IndexError as e: # e 被赋值为引发的实例对象... 在 try 处理器里，as 是可选的（省略的话，实例就简单地不被赋给任何名字），但包含它能允许处理器同时访问实例里的数据和异常类里的方法。这种模型对用类定义的自定义异常同样生效——例如下面这段代码，把构造参数传给异常类，处理器就能通过被赋值的实例拿到这些参数：python class MyExc(Exception): pass try: raise MyExc('oops') # 给异常类传构造参数 except MyExc as e: # 处理器里可访问实例属

性`print(e.args)` # 打印 ('oops',) 不过，这已经几乎要侵入下一章的主题了，其余细节我们留到那时再说。无论来源如何，异常永远是靠类实例（instance）对象来识别的，而且任意时刻至多有一个异常处于活跃状态（第 35 章的 `except` 分组除外）。一旦被程序中任何一句 `except` 捕捉，异常即告终结、不再向别的 `try` 传播——除非它被另一条 `raise` 语句或错误再次引发。

深度理解

- 核心概念：`except ... as X` 是一条钩子（hook）——它把异常实例“钩”到变量上，让我们能读取异常携带的数据。这相当于把“捕捉异常”从“只看类型”升级为“同时拿到对象”。
- 底层实现：`except Exception as e:` 被编译为先把 `e` 绑定到当前异常实例，再进入处理器字节码。这个绑定是局部的（见下一节“作用域”），`e.args` 实际上是 `BaseException.args` 属性，即传给异常构造函数的位置参数元组。
- 设计的理由：异常不只传递“冒了什么锅”，还应该携带信息（错误信息、原始数据）。`as` 让语言有机会把载体（实例）交给用户，却无需改变 `try/except` 的基本用法。
- 实际用途：生产代码中 `except ValueError as e: logger.error(f"bad value: {e}")`、`except json.JSONDecodeError as e: 读 e.doc/e.lineno` 定位 JSON 出错点，都是这个机制的典型用法。
- 新手误区：①以为 `as` 会赋异常错误信息字符串——其实绑定的是异常实例；②以为省略 `as` 就没有异常处理——

只是拿不到实例而已；③自定义异常的子类里重写 `init` 却没调用 `super().init(...)`，导致 `e.args` 为空——这是第 35 章的主战场。

代码分析

```
class MyExc(Exception): pass           # Exception
try:
    raise MyExc('oops')                # 'oops' args
except MyExc as e:                     # e
    print(e.args)                      # ('oops',)
```

·解释器如何通过钩子传递数据：`raise MyExc('oops')` 时构造器将 `'oops'` 存为 `args` 属性；异常传播到 `except MyExc` 匹配后，字节码执行栈绑定 `e` 到该实例；`e.args` 读取到 `('oops',)`。注意 `raise MyExc('oops')` 中 `raise` 后面可跟 `MyExc(...)` 传参——再次证明了“传实例”型 `raise` 最常见。

·设计思想：`as` 把“异常处理”从“类型上分诊”推向“数据级处理”——既保持了类型匹配的快速，又提供了上下文侦察的手段。这是把“异常对象”设计为“携带信息的载体”的直接收益：类型决定路由，数据决定诊断。

34.7.3 Scopes and except as (作用域与 except as)

英文原文

We'll study exception objects in more detail in the next chapter. Now that we've seen the `as` variable in act

ion, though, we can finally clarify the related scope issue summarized back in Chapter 17. As mentioned in that chapter, the variable used to access an exception in the `as` clause of an `except` is localized to the `except` block—the variable is not available after the block exits, much like a temporary loop variable in comprehension expressions:

```
text $ python3 >> try:
```

```
... 1 / 0 ... except Exception as X: # The "as" localizes names to except block ... print(X) ... division by zero >
> X NameError: name 'X' is not defined
```

Unlike comprehension loop variables, though, this variable is removed after the `except` block exits. This is done because the variable would otherwise retain a reference to the runtime call stack, which would defer garbage collection and thus retain excess memory space. This removal occurs, though, even if you're using the name for other purposes in the surrounding scope, and is a much more extreme policy than that used for comprehensions:

```
text >> X = 99 >> {X for X in 'hack'} # Comprehensions localize but don't remove {'a','c','k','h'} >> X 99 >>
X = 99 >> try:
```

```
... 1 / 0 ... except Exception as X: # But "as" localizes and removes on exit! ... print(X) ... division by zero >>
X # Where did my X go? – an odd boundary case NameError: name 'X' is not defined
```

中文翻译

下一章我们会更详细地研究异常对象。不过，既然现在我们已经见识了 `as` 变量的实际样子，就可以终于澄清第 17 章里概括过的那个相关作用域问题。正如那章提到的，在 `except` 的 `as` 子句中用来访问异常的变量，被局部化 (localized) 到 `except` 块之内——变量在代码块结束后就不可用了，这与面向推导式 (comprehension) 中的临时循环变量非常相似：

```
text $ python3 >>> try: ... 1 / 0 ... except
Exception as X: # as 把名字局域化到 except 块内... print
(X) ... division by zero >>> X
NameError: name 'X' is not defined
```

不过，与推导式的循环变量不同，这个变量会在 `except` 块退出时被删除 (removed)。之所以这样做，是因为如果不删除，这个变量就可能一直保留对运行时调用栈 (runtime call stack) 的引用，从而推迟垃圾回收 (garbage collection)、白白占用多余内存。而且——这里就会出现一个非常奇怪的边界情况——即便你在外层作用域用同一个名字存了别的值，删除照样会发生，这比推导式那种“局域化但不删除”的策略极端得多：

```
text >>> X = 99 >>> {X for X in 'hack'} # 推导式：局域化但不删除{'a','c','k','h'}
>>> X
99 >>> X = 99 >>> try: ... 1 / 0 ... except
Exception as X: # 但 "as" 既局域化又删除！ ... print(X) ...
division by zero >>> X
# 我的 X 哪里去了？——一个奇怪的边界情况
NameError: name 'X' is not defined
```

正因如此，你通常应该在 `try` 语句的 `except` 子句里使用独特的变量名——即便它们的名字实质上已被局域化 (localized) 也一样。如果 `try` 语句结束后你确实还需要引用异常实例，

```
那就把它赋给另一个不会被自动删除的名字: text >>> try
: ... 1 / 0 ... except Exception as X: # Python 会删除这个
引用... print(X) ... saveit = X # 需要时把异常赋给别的名字来保留它...
division by zero >>> X NameError: name
'X' is not defined >>> saveit ZeroDivisionError('division by zero',)
```

深度理解

- 核心概念: `except ... as X` 绑定的名字是块级局域变量, 且在 `except` 块退出时被显式删除。这与推导式 (comprehension) 变量的"局域化但不删除"策略形成鲜明对照——前者更激进, 因为异常实例往往持有 `traceback` 引用链。
- 底层实现: CPython 在 `except` 块退出时对 `as` 变量执行 `DELETERNAME/DELETERFAST` 指令, 同时显式清理未完成的轨迹。根本动机: 异常实例的 `traceback` 属性引用着整个运行时调用栈的帧对象, 若不删除, 会让整条栈链 (每帧又引用局部字典、代码对象) 滞留内存无法回收。
- 设计原因: 这是一次"内存优先于便利"的取舍。推导式删除名字只影响循环变量, 异常名字删除则是一个内存泄漏 (cycle) 维护问题的务实解。正如原书所说, 宁愿让你再取一个名字把异常存下来, 也不愿让回收期无限推迟。
- 实际问题: REPL/脚本里最常见的出坑现场——在 `except` 之外想用 `e` 拿异常信息, 结果 `NameError`。正确姿势是提前 `err = e` 转存, 或者用第 26 章的 `raise ... from` 等方式保留上下文。

·初学者误区：①误以为 `except` 里绑定变量与普通局部变量一致，块外也能用；②不知道删除机制，导致调试时 `print(e)` 报 `NameError` 就怀疑是版本 bug；③在推导式里被坑过后，误以为 `except` 也是"局域化可按需访问"。

代码分析

```
# as
try:
    1 / 0
except Exception as X:           # X
    print(X)                     # division by zero
# X
saveit = X                       # NameError

try:
    1 / 0
except Exception as X:
    print(X)
    keep = X                     #
# keep X
print(keep)                      # ZeroDivisionError(...)
```

·解释器如何处理：每个 `except` 块的编译单元带一个特殊的异常变量槽位。解释器进入处理器时把异常实例绑定到该槽位（即 `asio`），退出块时执行隐式删除指令 `DELETE FAST`，把名字从局部命名空间抹去。因此块外的任何访问都必然 `NameError`——这与推导式使用独立的"隐藏变量"完全不同：推导式只是换了个临时名字，留着外层名字不动。

·设计思想：这个语义是"显式优于隐式"的又一体现——想要保留数据？请显式把它复制到另一个名字。语言宁可删

除也不猜你的下一步，避免"想当然还能用"的坑。

34.7.4 Propagating Exceptions with raise (用 raise 传播异常)

英文原文

The raise statement is a bit more feature-rich than we've seen thus far. For example, a raise that does not list an exception to raise simply reraises the currently active exception. This form is typically used if you need to catch and handle an exception but don't want the exception to die in your handler:

```
text >>> try: ... raise IndexError('code') # Exceptions remember arguments ... except IndexError: ... print('propagating') ... raise # Reraise most recent exception ..  
. propagating Traceback (most recent call last): File "<stdin>", line 2, in <module> IndexError: code
```

Running a raise this way reraises the exception and propagates it to a higher handler (or the default handler at the top, which stops the program with a standard error message). Notice how the argument we passed to the exception class shows up in the error messages; you'll learn why this happens in the next chapter.

中文翻译

raise 语句比我们迄今所见的要更丰富一点。例如，一条没有列出要引发异常的 raise 语句，只是简单地重新引发 (re-raise) 当前活跃的异常。当你需要“捕捉并处理”某个异常、却不想让该异常死在你的处理器里时，典型就用这种形式：

```
text >>> try: ... raise IndexError('code') # 异常会记住它的参数... except IndexError: ... print('propagating') ... raise # 重新引发最近的异常... propagating
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
IndexError: code
这样运行 raise 会把异常重新引发，并传播到更高层的处理器（或顶层的默认处理器——它会用一条标准错误消息终止程序）。注意我们传给异常类的参数是如何出现在错误消息里的；下一章你会学到为什么是这样的。
```

深度理解

·核心概念：裸 raise = “重抛当前异常”。它让处理器能“看一眼异常、记个好日志，然后照样让它继续往上抛”。这是传播 (propagation) 语义的核心动作。

·底层实现：raise 无参数形式编译为 RAISEVARARGS0 指令，它从线程状态 (thread state) 的异常槽取出当前活跃异常，重新进入传播流程（再次遍历调用栈找 except）。注意它不再走一次“构造异常”。

·设计思想：传播是异常机制的“以太”——异常不该在默默吞下它的处理器里悄然死亡（那样常掩盖 bug）。裸 raise 给了 “logging/observe + re-raise” 的一行解决方案，

也方便"审计包装层"。

·实际问题：框架代码（Web 中间件、ORM 外层）常在 `except` 里做统一日志（`logger.error(...)`）再用 `raise` 重新抛出，让上层真正对错误负责；一旦漏掉 `raise`，错误将被静默吞没，排查难上加难。

·初学者误区：①在 `except` 之外调用裸 `raise` → `Runtime Error: No active exception to re-raise`；②在 `except` 里写了 `pass` 而不 `raise`，导致错误被"吞掉"，程序后续逻辑在错误状态下继续；③把裸 `raise` 与 `raise e` 混为一谈：前者保留原 `traceback` 继续传播，后者在处理器内会以"新异常"面貌出现（`context` 隐式入链，显示 "During handling..." 横幅），两者错误显示的观感完全不同。

代码分析

```
try:
    raise IndexError('code')      #
except IndexError:                #
    print('propagating')         #
    raise                         # raise
```

·解释器如何处理：1) `raise IndexError('code')` 构造异常实例并进入异常机制；2) 很快被同函数的 `except IndexError` 匹配，处理器执行 `print`；3) 裸 `raise` 指令把活跃异常实例取出、保留其原始 `traceback` 链，向上一层跳转——于是 REPL 里最终打印 `IndexError: code` 的 `traceback`。若再往上无人捕捉，解释器调用默认异常处理器打印完整栈后退出。

·设计思想：异常实例"记得自己的参数"（`code`），体现

异常对象的唯一目的之一——携带诊断信息。这既是第 3 5 章内容的预告，也让裸 raise 的重抛行为可以保留完整诊断链。

34.7.5 Exception Chaining: raise from (异常链: raise from)

英文原文

Exceptions can sometimes be triggered in response to other exceptions—both deliberately and by new program errors. To support full disclosure in such cases, Python also allows raise statements to have an optional from clause: raise newexception from otherexception When the from is used in an explicit raise request, the expression following from specifies another exception class or instance to attach to the cause attribute of the new exception being raised. If the raised exception is not caught, Python prints both exceptions as part of the standard error message:

```
text >> try:
... 1 / 0 ... except Exception as E: ... raise TypeError('Bad') from E # Explicitly chained exceptions ...
Traceback (most recent call last): File "<stdin>", line 2, in <module> ZeroDivisionError: division by zero
The above exception was the direct cause of the following exception:
Traceback (most recent call last): File "<stdin>", l
```

ine 4, in <module> TypeError: Bad

When an exception is raised implicitly by a program error inside an exception handler, a similar procedure is followed automatically: the previous exception is attached to the new exception's context attribute and is again displayed in the standard error message if the exception goes uncaught:

```
text >> try:
```

```
... 1 / 0 ... except: ... badname # Implicitly chained exceptions ...
Traceback (most recent call last): File "<stdin>", line 2, in <module>
ZeroDivisionError: division by zero
During handling of the above exception, another exception occurred:
Traceback (most recent call last): File "<stdin>", line 4, in <module>
NameError: name 'badname' is not defined
```

In both cases, because the original exception objects thus attached to new exception objects may themselves have attached causes, the causality chain can be arbitrarily long, and is displayed in full in error messages. That is, error messages might give more than two exceptions. The net effect in both explicit and implicit chaining contexts is to allow programmers to know all exceptions involved when one exception triggers another:

```
text >> try:
```

```
... try: ... raise IndexError() ... except Exception as E: ...
```

raise TypeError() from E ... except Exception as E: ... raise SyntaxError() from E ... Traceback (most recent call last): File "<stdin>", line 3, in <module> IndexError The above exception was the direct cause of the following exception: Traceback (most recent call last): File "<stdin>", line 5, in <module> TypeError The above exception was the direct cause of the following exception: Traceback (most recent call last): File "<stdin>", line 7, in <module> SyntaxError: None

Code like the following similarly displays three exceptions, though implicitly triggered by handler errors (its separator lines are "During handling of the above exception..." instead of "The above exception was the direct cause..."):

```
python try: try: 1 / 0
except: badname
except: open('nonesuch')
```

Exception chains impact error displays, but do not affect the way that exceptions are named and caught in try statements: chains are simply recorded in exception-object attributes which may be inspected as usual where useful.

Like the combined try, chained exceptions are similar to utilities in other languages (including Java and C#) though it's not clear which languages were borrowers. In Python, it's not unusual to see exception chains in

n error messages, but it is uncommon to create them explicitly with `raise`, so we'll defer to Python's manuals for more details.

As a footnote on this topic, though, Python also provides a way to stop exceptions from chaining: a `raise from None` allows the display of the chained exception context to be disabled when needed. This makes for less cluttered error messages in applications that convert between exception types while processing exception chains.

中文翻译

异常有时会因其他异常而起——既有故意的，也有新程序错误引发的。为了在这些情况下支持“充分披露”（full disclosure），Python 也允许 `raise` 语句带一个可选的 `from` 子句：`raise newexception from otherexception` 在显式 `raise` 请求中使用 `from` 时，`from` 后面的表达式指定另一个异常类或实例，它会挂到新引发异常的 `cause` 属性上。如果被引发的异常没有被捕捉，Python 会把两个异常都打印出来，作为标准错误消息的一部分：`text >>> try: ... 1 / 0 ... except Exception as E: ... raise TypeError('Bad') from E # 显式链接 (chain) 异常... Traceback (most recent call last): File "<stdin>", line 2, in <module> ZeroDivisionError: division by zero The above exception was the direct cause of the following exception: Traceback (most recent call last): File "<stdin>", line 4, in <module> TypeError: Bad 当异常被隐式引发——即由异常处理器内部的程序错误引发——时，会自动遵循类似的流`

程：前一个异常被挂到新异常的 context 属性上，如果新异常最终没被捕捉，它同样会出现在标准错误消息中：

```
>>> try: ... 1 / 0 ... except: ... badname # 隐式链接的异常...
Traceback (most recent call last): File "<stdin>", line 2, in <module> ZeroDivisionError: division by zero
During handling of the above exception, another exception occurred:
Traceback (most recent call last): File "<stdin>", line 4, in <module> NameError: name 'badname' is not defined
```

在两种情况下，因为被挂到新异常对象上的原始异常对象本身也可能带着它的起因（cause），所以因果链（causality chain）可以任意长，并会在错误消息中完整显示。也就是说，错误消息可能给出不止两个异常。显式与隐式链接的净效果都是一样的：当一个异常引发另一个异常时，程序员能看到所有涉事的异常：

```
>>> try: ... try: ... raise IndexError() ... except Exception as E: ... raise TypeError() from E ... except Exception as E: ... raise SyntaxError() from E ...
Traceback (most recent call last): File "<stdin>", line 3, in <module> IndexError
The above exception was the direct cause of the following exception:
Traceback (most recent call last): File "<stdin>", line 5, in <module> TypeError
The above exception was the direct cause of the following exception:
Traceback (most recent call last): File "<stdin>", line 7, in <module> SyntaxError: None
```

像下面这样的代码同样会显示三个异常，只不过它们是由处理器错误隐式触发的（它的分隔行是"During handling of the above exception..."（处理上述异常时，又发生了另一个异常），而不是"The above exception was the direct ca

use..." (上述异常是下面异常的直接起因)) : `python try : try: 1 / 0 except: badname except: open('nonesuch')` 异常链影响的是错误的显示, 但不影响 `try` 语句中对异常命名与捕捉的方式: 链只是被记录在异常对象的属性里, 需要时照常查看即可。与组合 `try` 一样, 链式异常与其他语言 (包括 Java 和 C#) 的类似工具异曲同工——虽然并不清楚到底是谁借鉴了谁。在 Python 中, 错误消息里看到异常链并不罕见, 但显式用 `raise` 制造链却不常见, 所以更详细的说明就交给 Python 官方手册吧。作为这个话题的脚注: Python 还提供了一种阻止异常链接的方法——`raise from None` 可以在需要时禁用链式异常上下文的显示。这对于在异常处理链中做异常类型转换的应用程序来说, 能让错误消息更清爽、不杂乱。

深度理解

- 核心概念: 异常链 (chaining) 回答"这个异常从何而来": 显式用 `from` 指定 `cause`; 隐式由处理中的新错误自动记入 `context`。链可以多级、可以任意长, 全部会显示在 `traceback` 里。
- 底层实现: `raise X from Y` 时解释器设置 `X.cause = Y` 并清除 `X.suppresscontext`; 处理器里抛新错误则把当前活跃异常记入 `context`。 `traceback` 打印逻辑先看 `cause` ("direct cause" 横幅), 无 `cause` 才看 `context` ("During handling" 横幅), `suppresscontext` 为真则都不显示——`raise from None` 正是设置这个开关。
- 设计原因: "充分披露 (full disclosure)" 是调试哲学的直接产物: 只告诉你最后的 `TypeError: Bad`, 你根本不知

道根因是 `ZeroDivisionError`。把因果链完整呈现，才能让"异常引发的异常"不再是诊断盲区。

·实际问题：三层库调用里一个 `ValueError` 背后常藏着 `KeyError`；日志系统、RPC 网关做异常转换时，`from` 是保留根因的标准做法；而 `from None` 则用于故意隐藏内部细节、只对外暴露业务语义异常的边界接口。

·初学者误区：①看到两个 `traceback` 以为程序抛了两个异常——其实只有一个活跃异常，另一个是历史；②混淆 `cause`（显式、打印"direct cause"）与 `context`（隐式、打印"During handling"）；③在 `except` 里无脑 `raise e`（重新抛同一实例）而不是裸 `raise`，导致 `traceback` 被截断/链显示异常；④忘了 `from` 的语义是"挂接"而不是"替换"。

代码分析

```
#
try:
    1 / 0                                # ZeroDivisionError
except Exception as E:
    raise TypeError('Bad') from E        # __cause__ traceback...

#
try:
    1 / 0
except:
    badname                               # NameError __context__

#
try:
    try:
        raise IndexError()
```

```

except Exception as E:
    raise TypeError() from E
except Exception as E:
    raise SyntaxError() from E      # IndexError → TypeError...

#
try:
    1 / 0
except Exception as E:
    raise TypeError('Bad') from None  # __suppress_context__ ...

```

·解释器如何处理：raise TypeError('Bad') from E 的字节码序列：构造 TypeError 实例 → 取 E → 设置 exc.cause = E、exc.suppresscontext = False → 传播。未捕捉时，traceback 打印器先打印 ZeroDivisionError 的帧链，再打印横幅"上述异常是以下异常的直接起因"，接着打印 TypeError 的帧链。隐式链则完全由解释器在"处理器内再抛新异常"时自动完成 context 记录，打印器看到 cause 为空就用"During handling"横幅展示 context。

·设计思想：链式异常实现了"错误责任链"——每一层包装都保留下层现场。from None 则是一个精心设计的逃生门：语言尊重你"故意切断因果"的明确意图，也让你能控制对外暴露的信息粒度。

34.8 The assert Statement (assert 语句)

英文原文

As a somewhat special case for debugging purposes, Python also includes the assert statement in its excep

tions toolset. It is mostly just syntactic shorthand for a common raise usage pattern, and an assert can be thought of as a conditional raise statement. A statement of the form: `assert test, data` # The data part is optional works like the following code: `python if debug: if not test: raise AssertionError(data)` In other words, if the test evaluates to false, Python raises an exception: the data item (if it's provided) is used as the exception's constructor argument. Like all exceptions, the built-in `AssertionError` exception will kill your program if it's not caught with a `try`, and the data item shows up as part of the standard error message: `text >>> language = 'Java'>>> assert language.startswith('Py'), "You're using the wrong language!"` `AssertionError: You're using the wrong language!` As an added feature, assert statements are removed from a compiled program's bytecode—and hence not run—if the `-O` Python command-line flag is used to optimize the program. The `debug` flag is a built-in and unchangeable name that is automatically set to `True` unless the `-O` flag is used. When `debug` is `False` for `-O`, any code predicated on it being `True` is removed, including asserts. Hence, to disable (and omit) asserts, run code with a command line like `python -O file.py`, or generate optimized bytecode before program runs with similar options in the `compileall` standard-library module or `compile` built-in function. See Python's manuals for details on the module and function.

中文翻译

作为调试目的的一个特殊用例，Python 在异常工具箱里还包含 `assert` 语句。它基本只是一条常用 `raise` 用法的语法简写（syntactic shorthand），可以把 `assert` 想成一条条件式 `raise` 语句。形如：`assert test, data` # `data` 部分是可选的的语句，等价于下面这段代码：`python if debug: if not test: raise AssertionError(data)` 换句话说，如果 `test` 求值为假，Python 就会引发一个异常：`data` 项（如果提供了的话）会作为异常的构造参数。和所有异常一样，内置的 `AssertionError` 异常如果没被 `try` 捕捉，就会杀掉你的程序；`data` 项会作为标准错误消息的一部分显示出来：`text >>> language = 'Java' >>> assert language.startswith('Py'), "You're using the wrong language!" AssertionError: You're using the wrong language!` 作为一项附加特性，如果用 `-O` 命令行标志优化程序，`assert` 语句会从编译后的程序字节码中被删除——因此不会运行。`debug` 是一个内置的、不可修改的名字，除非用了 `-O` 标志，否则它自动被设为 `True`。当 `-O` 使 `debug` 为 `False` 时，任何以“它为真”为前提的代码都会被删除，其中就包括断言。因此，要禁用（并省略）断言，可以用 `python -O file.py` 这样的命令行运行代码，或者在程序运行前用标准库 `compileall` 模块或内置 `compile` 函数里的类似选项生成优化字节码。模块与函数的细节请参见 Python 手册。

深度理解

·核心概念：`assert test, data` $\equiv$ `if debug: if not test: raise AssertionError(data)`。它把“检查前置条件 $\rightarrow$ 失败抛

错"压缩成一行，且随 -O 开关整条消失——是开发期防线而非运行时防御。

·底层实现：debug 是一个由解释器编译时 (compile-time) 决定的常量——CPython 里编译单元直接按 debug 的真假来决定 assert 的字节码是否生成：-O 时 assert 根本不产生指令（等价于空语句），所以它"零成本"。

·设计原因：把断言与普通 if 区分开，正是为了给语言一个"编译期可整体移除的检查"——生产环境不需要付这笔检查的开销。代价是它不能用于真正的错误处理（那些必须总是运行的检查）。

·实际问题：教科书式的用法是"验证前提条件"（参数非负、配置合法、不变量保持），配合 AssertionError(data) 让失败信息带上值；而输入验证（用户传的数据）绝不该用 assert——因为 -O 下它会被静默删掉，防线直接消失。

·初学者误区：①把 assert 当 raise 的日常替身，用来做输入校验（-O 后全部失效）；②误以为 assert False 只是"打印警告"——它是实打实抛异常，没被捕捉就终止程序；③以为 debug 可以在运行时改（debug = False 会直接 SyntaxError——它是编译期常量）。

代码分析

```

assert test, data                                #
if __debug__:                                    # __debug__  True
    if not test:                                  # test
        raise AssertionError(data)               # data

language = 'Java'
assert language.startswith('Py'), "You're using the wrong language..."
# → → AssertionError: You're using the wrong language!

```

·解释器如何处理：不带 `-O` 编译时，`assert` 被编译成"求值 `test` → 若为假调用 `AssertionError(data)` 并抛出"的指令序列；带 `-O` 编译时，编译器看到 `debug` 为假，`assert` 语句整体退化为一条 `NOP` 或干脆不生成任何指令。`language.startswith('Py')` 在求值时返回 `False`，于是断言触发，REPL 打印 `AssertionError` 及构造参数。

·设计思想：断言是"程序的自我检查"——它假设"我的代码应该永远是对的"，只是预防万一。这种"信任自己的代码 + 极低开销"的定位，正是它与面向外部输入的 `if/raise` 校验之间的哲学分界线。

34.8.1 Example: Trapping Constraints (but Not Errors!) (示例：捕获约束条件——但不是错误！)

英文原文

Here's a less politically charged example of `assert` in action. Assertions are typically used to verify program conditions during development. When displayed, t

heir error message text automatically includes source code line information and the value listed in the assert statement. Consider the file `asserter.py` in Example 34-5: Example 34-5. `asserter.py`

```
python def f(x): assert x < 0, 'x must be negative'
```

return x * 2

Running this normally triggers the assertion error for positive numbers, but running with `-O` does not:

```
$ python3 >> import asserter >> asserter.f(-3) 9
>> asserter.f(3) Traceback (most recent call last): File
"<stdin>", line 1, in <module> File "/.../LP6E/Chapter
34/asserter.py", line 2, in f assert x < 0, 'x must be neg
ative'AssertionError: x must be negative
```

```
$ python3 -O >> import asserter >> asserter.f(3) 9
```

It's important to keep in mind that `assert` is mostly intended for trapping user-defined constraints, not for catching genuine programming errors. Because Python traps programming errors itself, there is usually no need to code `assert` to catch things like out-of-bounds indexes, type mismatches, and zero divides:

```
python def reciprocal(x): assert x != 0 # A generally use-
less assert!
```

```
return 1 / x # Python checks for zero automatically
```

Such `assert` use cases are usually superfluous—because Python raises exceptions on errors automatically, you might as well let it do the job for you. As a rule, y

ou normally don't need to do error checking explicitly in your own code.

Of course, there are exceptions to most rules. As suggested earlier in the book, if a function has to perform long-running or unrecoverable actions before it reaches the place where an exception will be triggered, you still might want to test for errors. Even in this case, though, be careful not to make your tests overly specific or restrictive, or you will limit your code's utility.

For another example of common assert usage, see the abstract superclass example in Chapter 29; there, we used assert to make calls to undefined methods fail with a message. It's a rare but useful tool.

中文翻译

这里有一个不那么“政治敏感”的 assert 实战示例。断言通常用来在开发期间验证程序条件。触发时，它们的错误消息文本会自动包含源码行信息以及 assert 语句中列出的值。

请看示例 34-5 中的文件 `asserter.py`：示例 34-5. `asserter.py`

```
python def f(x): assert x < 0, 'x must be negative'
# 断言：x 必须为负
return x 2 # 平方正常运行时，正数会触发断言错误；而用 -O 运行时不会：
text $ python3 >>> import asserter >>> asserter.f(-3) 9 >>> asserter.f(3)
Traceback (most recent call last): File "<stdin>", line 1, in <module>
File "/.../LP6E/Chapter34/asserter.py", line 2, in f assert x < 0, 'x must be negative'
AssertionError: x must be negative $ python3 -O >>> impor
```

t asserter >>> asserter.f(3) 9 请务必记住: assert 主要是用来捕获用户定义的约束条件的, 而不是用来捕获真正的编程错误。因为编程错误由 Python 自己捕获, 通常没必要用 assert 去抓越界下标、类型不匹配、除零这类事情: python def reciprocal(x): assert x != 0 # 一条通常毫无用处的 assert! return 1 / x # Python 会自动检查除零这类 assert 用例通常是多余的——因为 Python 在出错时会自动引发异常, 你不妨把这份工作交给它。作为一条规则: 你通常不需要在自己的代码里做显式的错误检查。当然, 大多数规则都有例外。正如本书前面所建议的: 如果一个函数在到达异常触发点之前, 必须先执行长时间运行或不可恢复的动作, 你也许仍想自己测试错误。不过即便在这种情况下, 也要小心别把测试做得过于具体或苛刻, 否则会限制你代码的用途。另一个常见的 assert 用法示例, 见第 29 章的抽象超类示例——在那里我们用 assert 让"调用未定义的方法"带着消息失败。这是一种罕见但有用的工具。

深度理解

·核心概念: assert 的正确打开方式是"开发期约束": 参数不变量、抽象方法未实现、算法前提——而不是替代语言自带的错误检查。它的消息文本自动含源码行与断言语值, 便于快速定位。

·底层实现: asserter.py 里的 assert x < 0 触发时, AssertionError 的 traceback 直接把源码行 assert x < 0, 'x must be negative' 显示出来——因为 traceback 里的 linecache 会去读源文件对应行, 所以"行信息自动包含"是免费的。-O 下 import 的模块是预先编译的优化字节码,

断言指令不存在，f(3) 直接返回 9。

·设计原因：作者特意强调"断言捕获约束、不捕获错误"，是为了纠正一个流传甚广的坏习惯——把 assert 当运行时防御。这既关乎 -O 的语义，也关乎代码的健壮性预期：断言是"测试我的推理"，异常是"处理外部世界的意外"。

·实际问题：把"昂贵操作之前的预检"留给 assert（如数据库连接池断开检查、缓存刷新前状态校验），把"用户输入校验"交给 if/raise ValueError；测试框架（pytest）里 assert 是断言基础设施，但生产代码 -O 部署时断言全部失效，所以"必须生效"的检查永远用 if。

·初学者误区：①用 assert 校验用户输入（-O 生产环境直接失效，安全边界被撕开）；②assert x != 0 这种"Python 反正会报 ZeroDivisionError"的自我安慰式断言；③在 -O 下调试"为什么断言不触发"——先确认自己的启动参数。

代码分析

```
def f(x):
    assert x < 0, 'x must be negative'      #
    return x ** 2                           #

# __debug__ = True
# f(-3) → 9
# f(3) → AssertionError: x must be negativetraceback ...

# -O __debug__ = False
# → f(3) 9
```

```
def reciprocal(x):  
    assert x != 0      # Python  ZeroDivisionError  
    return 1 / x      #
```

·解释器如何处理：正常模式下 `assert x < 0` 编译为"比较 + 条件抛错"序列，`f(3)` 时触发 `AssertionError('x must be negative')`，异常沿调用栈传到 REPL 顶层，`traceback` 显示 `asserter.py` 第 2 行并附源码行文本。`-O` 模式下编译产物中该指令不存在——这解释了为什么 `f(3)` 静默返回 9：断言不是被跳过，而是根本没进字节码。

·设计思想：这个示例的教学顺序很讲究：先用"正数翻车、负数过关"展示断言机制，再用"除零断言"展示何时不该用，最后用"长耗时动作前的预检"给出例外场景——一套完整的"断言使用纪律"。

34.9 The with Statement and Context Managers (with 语句与上下文管理器)

英文原文

In addition to the tools we've seen so far, Python includes another that delegates exception-related tasks to objects. The `with` statement is designed to work with context manager objects that support a method-based protocol. The combination is similar in spirit to the way that iteration tools like `for` work with methods of the iteration protocol. The `with` statement is also similar to a "using" statement in the C# language. Alth

ough a somewhat optional and advanced tools-oriented topic (and once a candidate for the next part of this book), context managers are lightweight and useful enough to group with the rest of the exception tocolset here. In short, the `with` statement is designed to be an alternative to a common `try/finally` usage pattern: like that statement, `with` is in large part intended for specifying termination-time or "cleanup" activities that must run regardless of whether an exception occurs during the execution of a block of code. Unlike `try/finally`, though, the `with` statement is based upon a method-call protocol for specifying actions to be run around a block of code. This makes `with` less general, qualifies it as redundant in termination roles, and requires coding classes for objects that don't support its protocol. On the other hand, `with` also handles entry actions, can reduce code size where supported, and allows code contexts to be managed with full OOP. Python enhances some built-in tools with context managers, such as files that automatically close themselves, thread locks that automatically lock and unlock, and `async-function` tools that automatically await results per Chapter 20, but programmers can code their own context managers with classes too. Let's take a brief look at the statement and its implicit protocol.

中文翻译

除了我们已经见过的工具之外，Python 还有一种能把异常相关的任务委托给对象的工具。with 语句就是设计来与上下文管理器（context manager）对象配合使用的，这些对象支持一个基于方法（method-based）的协议。这种组合在精神上与 for 之类的迭代工具通过迭代协议的方法工作如出一辙。with 语句也与 C# 语言中的 using 语句类似。虽然这是一个多少有些“可选的高级工具”话题（还一度是本书下一部分的候选内容），但上下文管理器足够轻量也足够有用，因此在这里与其余的异常工具归为一组来讲解。简而言之，with 语句被设计成一条常见的 try/finally 惯用法（idiom）的替代品：与那条语句一样，with 在很大程度上是为了指定终止时刻或“清理（cleanup）”活动——这些动作无论代码块执行期间是否发生异常，都必须运行。与 try/finally 不同，with 语句建立在方法调用协议之上，用来规定在代码块周围运行哪些动作。这使 with 更不通用、在“终止性角色”中显得冗余，并且要求为不支持该协议的对象编写类。另一方面，with 也处理“进入”（entry）动作，在受支持的地方能减少代码量，并允许用完整的 OOP 来管理代码上下文。Python 为一些内置工具增强了上下文管理器，例如自动关闭自身的文件、自动加锁与解锁的线程锁、以及按第 20 章所述会自动 await 结果的异步函数工具；但程序员也可以用类编写属于自己的上下文管理器。让我们快速看一下这条语句及其隐式协议。

深度理解

·核心概念：with 是一条以对象为中心的异常工具：它把“进入动作 + 块 + 退出动作（无论成败）”打包交给上下文管理器对象。它与 for 之于迭代协议的关系同构——for

用迭代协议，with 用上下文管理协议。

·底层实现：with EXPR as VAR: 的字节码是：调用 EXPR.enter() 并把返回值赋给 VAR（若有 as）→ 执行 with-block → 退出时统一调用 EXPR.exit(type, value, tb)。块内抛异常时，exit 收到异常三件套（sys.excinfo() 的返回值），返回假则重新引发、真则掐灭。

·设计原因：作者点出 with 的两面性：它比 try/finally 更“专精”（少万能、少写三行、含进入动作、可 OOP 复用），但也更窄（只适用于实现协议的类）——这正是“工具各有取舍”章节主题的又一注脚。与 C# 的 using 对齐，说明这是现代语言共同的“RAII 式”资源管理追求。

·实际问题：最著名的加身者是文件对象——with open(...) as f: 自动 close，threading.Lock 自动 acquire/release，decimal.localcontext() 自动保存/恢复精度上下文。三者代表了“网络安全退出”最常见的三类资源：文件、锁、上下文状态。

·初学者误区：①以为 as 变量就是“表达式的结果”——其实它是 enter() 的返回值，两者常常不同（如 open 返回文件对象，而很多管理器返回 self）；②以为 with 能“吃掉”异常——默认 exit 返回假，异常照样往外抛，只有明确返回真才会吞掉；③为了 with 硬写协议类，用 try/finally 更合适时仍绕道。

代码分析


```

with open('somefile.txt') as myfile:      # open  __en...
    for line in myfile:                    #
        print(line)
# myfile close

import threading
lock = threading.Lock()                   # Lock
with lock:                                # acquire / release
    ...                                    # .....

import decimal
with decimal.localcontext() as ctx:        # decimal
    ctx.prec = 2                          #
    x = decimal.Decimal('1.00') / decimal.Decimal('3.00')

```

·解释器如何处理：with 语句被编译器展开为：先执行 open(...).enter() 拿到返回对象赋给 myfile，进入 for 循环；for 结束时或中途异常时，解释器走"终止理器"路径调用 myfile.exit(...)——文件对象在 exit 里执行 close()。线程锁同理：enter 调 acquire()，exit 调 release()。

·设计思想：with 不取代你的清理逻辑，而是把清理逻辑收纳进协议。作者强调"它处理进入动作、减少代码量、允许 OOP 管理代码上下文"三个优点，但也不避讳"冗余、复杂、有魔法成本"——这正是本书一贯的均衡视角。

34.9.1 Basic with Usage (with 基本用法)

英文原文

The basic format of the with statement looks like this
, with an optional part in square brackets here: python with expression [as variable]: with-block The statem

ents of the nested with-block are the main action to be run here. The expression is assumed to return an object that supports the context-management protocol (more on this protocol in a moment). This object may also return a value that will be assigned to the name variable if the optional as clause is present. Note that the variable is not necessarily assigned the result of the expression; the result of the expression is the object that supports the context protocol, and the variable may be assigned something else intended to be used inside the with-block. The object returned by the expression may then run startup code before the block is started, as well as termination code after the block is done—whether the block raised an exception or not. As noted, some built-in Python objects have been augmented to support the context-management protocol, and so can be used with the with statement. For example, file objects (covered in Chapter 9) have a context manager that automatically closes the file after the with block, regardless of whether an exception is raised, and regardless of if or when the version of Python running the code may close automatically. In abstract code: `python with open('somefile.txt') as myfile: for line in myfile: print(line)` Here, the call to `open` returns a simple file object that is assigned to the name `myfile`. We can use `myfile` with the usual file tools—in this case, the file iterator reads the file line by line in the for loop. However, the `open` result also s

upports the context-management protocol used by the `with` statement. After this `with` statement has run, the context-management machinery guarantees that the file object referenced by `myfile` is automatically closed, even if the `for` loop raised an exception while processing the file. Although file objects may also be closed automatically on garbage collection, it's not always straightforward to know when that will occur, especially when using alternative Python implementations. The `with` statement in this role is an alternative that allows us to be sure that the close will occur automatically after execution of a specific block of code. As covered earlier, we can achieve a similar effect with the more general and explicit `try/finally` idiom, but it requires three more lines of administrative code in this case (four instead of just one):

```
python myfile = open('somefile.txt')
try:
    for line in myfile:
        print(line)
finally:
    myfile.close()
```

Of course, we could skip both statements, but our file may not be closed if an exception is raised during the `for` loop, and this can matter in long-running programs (we'll revisit such trade-offs in Chapter 36). As another example, we won't cover Python's multithreading modules in this book, but the `lock` and `condition` synchronization objects they define may also be used with the `with` statement because they support the context-management protocol—in this case adding both entry and exit actions around a block. After importing `threading`:

```
python import threading
```

ing lock = threading.Lock() with lock: ... # access shared resources Here, the context-management machinery guarantees that the lock is automatically acquired before the block is executed and released once the block is complete, regardless of exception outcomes. Finally, the decimal module introduced in Chapter 5 also uses context managers to simplify saving and restoring the current decimal context, which specifies the precision and rounding characteristics for calculations: python with decimal.localcontext() as ctx: ctx.prec = 2 x = decimal.Decimal('1.00') / decimal.Decimal('3.00') After this statement runs, the current thread's context-manager state is automatically restored to what it was before the statement began. To do the same with a try/finally, we would need to save the context before and restore it manually after the nested block.

中文翻译

with 语句的基本格式如下，方括号中的部分是可选的（方括号只是标记，不是语法）：python with expression [as variable]: with-block 嵌套的 with-block 中的语句就是这里要运行的主动作。expression 被假定返回一个支持上下文管理协议（context-management protocol）的对象（一会儿就讲这个协议）。如果可选的 as 子句存在，这个对象还可以返回一个值，赋给名字 variable。注意：variable 不一定被赋值为 expression 的结果；expression 的结果是提供上下文管理的那个对象，而 variable 可能被赋值为别的、打算在 with-block 内部使用的东西。expression

返回的对象可以在块开始之前运行启动 (startup) 代码，也可以在块结束之后运行终止 (termination) 代码——无论块内是否引发了异常。如前所述，一些内置 Python 对象已经被增强以支持上下文管理协议，因此可以用于 with 语句。例如，文件对象（第 9 章讲过）有一个上下文管理器，会在 with 块结束后自动关闭文件——无论是否引发异常，也无论运行代码的那个 Python 版本是否会、以及何时自动关闭。抽象代码如下：`python with open('somefile.txt') as myfile: for line in myfile: print(line)` 这里，open 的调用返回一个简单的文件对象，赋给名字 myfile。我们可以用常规的文件工具来使用 myfile——在这个例子里，文件迭代器 (file iterator) 在 for 循环中逐行读取文件。不过，open 的结果也支持 with 语句使用的上下文管理协议。这条 with 语句运行结束后，上下文管理机制 (machinery) 保证被 myfile 引用的文件对象被自动关闭——即使 for 循环在处理文件时引发了异常也一样。虽然文件对象在垃圾回收时也可能被自动关闭，但要确切知道“什么时候发生”并不总是直截了当——尤其是在使用其他 Python 实现 (alternative Python implementations) 时。with 语句在此处的角色，就是一种替代方案：它让我们确定关闭动作会在某段特定代码执行完之后自动发生。如前面所述，用更通用、更显式的 try/finally 惯用法也能达到类似效果，但这里需要多写三行管理代码（总共四行，而不是一行）：`python myfile = open('somefile.txt') try: for line in myfile: print(line) finally: myfile.close()` 当然，我们也可以两条语句都省掉，但那样 for 循环中途引发异常时，我们的文件可能不会（及时）被关闭——这在长运行程序里可能会出问题（我们会在第 36 章再回访这种权衡）。再举一个例

子：本书不会涉及 Python 的多线程模块，但它们定义的锁（lock）和条件（condition）同步对象同样可以用于 with 语句，因为它们支持上下文管理协议——这一次是在代码块周围同时加上“进入”与“退出”动作。在导入 threading 之后：`python import threading lock = threading.Lock()` with lock: ... # 访问共享资源这里，上下文管理机制保证锁会在块执行前被自动获取（acquired）、块结束后被自动释放（released）——与异常的结局无关。最后，第 5 章介绍的 decimal 模块也使用上下文管理器，来简化“保存与恢复当前十进制上下文”的过程——这个上下文规定了计算的精度与舍入特性：`python with decimal.localcontext() as ctx: ctx.prec = 2 x = decimal.Decimal('1.00') / decimal.Decimal('3.00')` 这条语句运行结束后，当前线程的上下文管理器状态被自动恢复为语句开始前的样子。若用 try/finally 做同样的事，我们必须在嵌套块之前手动保存上下文、之后手动恢复。

深度理解

·核心概念：with 的格式一句话：expression 提供“入口/出口”协议对象，as variable 拿的是 enter 的返回值而非表达式本身。块 = 主行动，进出动作由管理器代管——“进入启动、退出收尾、成败不论”。

·底层实现：with expr as var: 编译展开为：var = expr.enter() → try: 块 → 退出时 exit(typ, val, tb)。文件对象的 enter 返回 self（文件自身），exit 调用 self.close()；锁的 enter/exit 分别调 acquire()/release()；decimal.localcontext() 的 enter 复制一份当前上下文、exit 恢复

原状。

·设计原因：书中给了三种原生用法（文件、锁、decimal 上下文）恰恰对应三类资源管理问题：①"何时关闭不确定"（跨实现 GC 差异）；②"进出必须配对"（锁）；③"状态要保存恢复"（上下文）。with 把这三类共性动作协议化，避免每次手写样板。

·实际问题：with open(...) as f 现在是文件操作的标准姿势；with lock 是线程安全的标准姿势；with decimal.localcontext() 是临时精度计算的标准姿势。此外自建类实现协议后，任何"有始有终"的组件（数据库连接、HTTP session）都能享受同一套便捷。

·初学者误区：①以为 as myfile 绑定的是"上下文管理器对象"而非 enter 返回值——在文件对象这里恰好相同，但用户自定义管理器可能返回完全不同的东西；②以为 with 只能用于文件；③以为 with 和 try/finally 完全等价——with 只对支持协议的对象有效，而 try/finally 对任何代码都行。

代码分析

```
with open('somefile.txt') as myfile:      # __enter__
    for line in myfile:                    #
        print(line)
# myfile close

import threading
lock = threading.Lock()
with lock:                                # acquire
    ...                                    #
# release
```

```
import decimal
with decimal.localcontext() as ctx:      # decimal
    ctx.prec = 2                          #
    x = decimal.Decimal('1.00') / decimal.Decimal('3.00')
# save/restore
```

·解释器如何处理：三条 with 的执行模型一致：先执行 enter 并绑定 as 变量 → 运行块 → 块结束（无论正常、break、return 还是异常）都触发 exit。异常情形下 exit 收到异常三件套，文件对象返回 None（假值）→ 异常继续传播。三者的 enter 返回值不同（文件=自身、锁=None、localcontext=上下文对象），充分演示"as 变量 ≠ 表达式结果"。

·设计思想：同样的语句、同样的协议、三种截然不同的资源语义——这就是"协议化"的威力：语言只定义四个钩子（两个方法 + 参数与返回值约定），把具体的进入/退出逻辑全部交给对象。与 for 复用迭代协议、+ 复用算术协议如出一辙。

34.9.2 The Context-Management Protocol (上下文管理协议)

英文原文

Although some built-in types come with context managers, we can also write new ones of our own. To implement context managers, classes use special methods that fall into the operator-overloading category to t

ap into the with statement. The interface expected of objects used in with statements is somewhat complex, and most programmers only need to know how to use existing context managers. For tool builders who might want to write new application-specific context managers, though, let's take a quick look at what's involved.

Here's how the with statement actually works: 1. The expression is evaluated, resulting in an object known as a context manager that must have enter and exit methods. 2. The context manager's enter method is called. The value it returns is assigned to the variable in the as clause if present, or simply discarded otherwise. 3. The code in the nested with block is executed. 4. If the with block raises an exception, the context manager's exit(type, value, traceback) method is called with the exception details. These are the same three values returned by sys.excinfo, described in the Python manuals and later in this part of the book. If this method returns a false value, the exception is reraised; otherwise, the exception is terminated. The exception should normally be reraised so that it is propagated outside the with statement after exit returns. 5. If the with block does not raise an exception, the exit method is still called, but its type, value, and traceback arguments are all passed in as None, and its return value is ignored.

Let's look at a quick demo of the protocol in action. The file `withas.py` in Example 34-6 defines a context-manager object that simply traces the entry and exit of the `with` block in any `with` statement it is used for.

Example 34-6. `withas.py`

```
python "A context manager that traces entry and exit
of any with statement's block"
```

```
class TraceBlock:
    def message(self, arg):
        print('running' + arg)
    def enter(self):
        print('[starting with block]')
        return self
    def exit(self, exctype, excvalue, exctb):
        if exctype is None:
            print('[exited normally]\n')
        else:
            print(f'[propagating exception: {exctype}]')
        return False
if __name__ == '__main__':
    with TraceBlock() as action:
        action.message('test 1')
```

```
print('reached')
with TraceBlock() as action:
    action.message('test 2')
```

```
raise TypeError
print('not reached')
```

Notice that this class's `exit` method returns `False` to propagate the exception; deleting the `return` statement would have the same effect, as the default `None` return value of functions is `false` by definition, but explicit is generally better in coding. Also notice that the `enter` method returns `self` as the object to assign to the `as` variable; in other use cases, this might return a completely different object instead.

When run, this module's self-test code uses its conte

xt manager to trace the entry and exit of two with statement blocks. The net effect automatically invokes the manager's enter and exit methods:

```
text $ python3 withas.py [starting with block] running test 1 reached [exited normally] [starting with block] running test 2 [propagating exception: <class'TypeError'>] Traceback (most recent call last): File "/.../LP6E/Chapter34/withas.py", line 25, in <module>
```

```
raise TypeError TypeError
```

Context managers can also utilize OOP state information and inheritance, but are somewhat advanced devices meant for tool builders, so we'll skip additional details here. See Python's standard manuals for the full story—including its coverage of the contextlib standard module that provides additional tools for coding context managers.

Also remember that the try/finally combination provides support for termination-time activities too, and is generally sufficient in roles that don't warrant coding classes to support the with statement's protocol.

中文翻译

虽然一些内置类型自带上下文管理器，我们也可以编写自己的新管理器。要实现上下文管理器，类要使用运算符重载（operator-overloading）类别的特殊方法，来接入 with 语句。with 语句期望的对象接口相当复杂，而大多数程序员只需要知道如何使用现成的上下文管理器。不过，对于那

些想编写应用特定 (application-specific) 上下文管理器的工具构建者，让我们快速看一下涉及的内容。下面是 with 语句实际的工作方式：1. 对表达式求值，得到一个称为上下文管理器 (context manager) 的对象，它必须拥有 enter 和 exit 方法。2. 调用上下文管理器的 enter 方法。它返回的值会赋给 as 子句中的变量（如果有的话），否则就被丢弃。3. 执行嵌套的 with 块中的代码。4. 如果 with 块引发了异常，就带着异常细节调用上下文管理器的 exit(type, value, traceback) 方法。这三个值与 sys.excinfo 返回的三个值相同，Python 手册和本书后面会讲到。如果该方法返回假值，异常被重新引发；否则，异常被终结。通常应该重新引发异常，以便 exit 返回后异常能传播出 with 语句。5. 如果 with 块没有引发异常，exit 方法仍会被调用，但它的 type、value、traceback 参数全都以 None 传入，且它的返回值被忽略。让我们快速演示一下协议的实际运作。示例 34-6 中的文件 withas.py 定义了一个上下文管理器对象，它简单地在所用的任意 with 语句中"记录"with 块的进入与退出。示例 34-6. withas.py python "一个记录任意 with 语句块进入与退出的上下文管理器" class TraceBlock : def message(self, arg): print('running' + arg) def enter(self): print('[starting with block]') # 打印"开始 with 块" return self # 返回 self 给 as 变量 def exit(self, exctype, excvalue, exc_tb): if exctype is None: print('[exited normally]\n') # 无异常：正常退出 else: print(f'[propagating exception: {exctype}]') # 有异常：打印并重抛 return False # 返回假 → 异常继续传播 if name == 'main': with TraceBlock() as action: action.message('test 1') # 正常运行 print('reached') with TraceBlock() as action: actio

`n.message('test 2') raise TypeError # 块内抛异常print('not reached') # 不会执行`注意这个类的 `exit` 方法返回 `False` 以传播异常；删掉这条 `return` 语句效果相同——因为函数默认的 `None` 返回值按定义就是假值——但编码时显式通常更好。还要注意 `enter` 方法把 `self` 作为要赋给 `as` 变量的对象返回；在别的用例中，它可能返回一个完全不同的对象。运行时，这个模块的自测代码用它的上下文管理器记录两个 `with` 语句块的进入与退出。净效果是自动调用管理器的 `enter` 和 `exit` 方法：
`text $ python3 withas.py [starting with block] running test 1 reached [exited normally] [starting with block] running test 2 [propagating exception: <class'TypeError'>] Traceback (most recent call last): File "/.../LP6E/Chapter34/withas.py", line 25, in <module> raise TypeError TypeError`上下文管理器还可以利用 OOP 的状态信息与继承，但它们是为工具构建者准备的、稍微高级的装置，所以我们在这里略过更多细节。完整故事请见 Python 官方手册——包括它对 `contextlib` 标准模块的介绍，该模块提供了编写上下文管理器的额外工具。也请记住：`try/finally` 组合同样支持“终止时刻活动”，而且在那些不值得为支持 `with` 语句协议而编写类的场景里，它通常已经够用。

深度理解

·核心概念：上下文管理协议 = 两个魔法方法：`enter`（返回给 `as` 的值）与 `exit(type, value, tb)`（接收异常三件套、用返回值决定“是否吞掉异常”）。这是本章最接近“框架”的知识点。

·底层实现：with 在编译期把块包进"终止处理器"中：exit 的调用参数直接来自异常机制的 sys.excinfo() 三件套；无异常时三个参数都是 None。返回值语义：真 → 解释器清除异常（吞掉）；假 → 重新引发。这正是 with 内部"伪装"成 try/except/finally 的骨架。

·设计原因：把"退出动作与异常处置"设计成一个方法而不是两条语句，是为了给工具作者钩子：不仅能做清理，还能选择"吞掉还是传播"。return False 的默认倾向（显式优于隐式）保证了"除非作者明确说吞，否则异常一定往外走"。

·实际问题：编写数据库事务管理器（成功才 commit、失败 rollback）、临时环境变量、性能计时器、redirectstdout 等，都是 enter/exit 的经典场景；contextlib.contextmanager 用生成器把两方法压成一次 yield，是官方推荐的简写。需要"处理异常"的场合尤其要注意 exit 的参数和返回值。

·初学者误区：①把 exit 的参数名写错导致 TypeError（必须三个位置参数）；②以为 exit 返回 True 才算"正常工作"——默认要返回假（或 None），否则异常被静默吞掉；③忘记 enter 必须返回对象，as 变量拿到 None 时一脸懵；④把 with 当作异常处理的替代品——它只负责"进出"，异常处置权在 exit 返回值上。

代码分析

```

class TraceBlock:
    def message(self, arg):
        print('running ' + arg)

    def __enter__(self):
        print('[starting with block]')      #
        return self                        # as self

    def __exit__(self, exc_type, exc_value, exc_tb):
        if exc_type is None:
            print('[exited normally]\n')    # None →
        else:
            print(f'[propagating exception: {exc_type}]')
            return False                    # →

with TraceBlock() as action:               # 1
    action.message('test 1')
    print('reached')                       #

with TraceBlock() as action:               # 2
    action.message('test 2')
    raise TypeError                        # __exit__ False...
    print('not reached')                   #

```

·解释器如何处理：第一次 with：enter 打印 [starting with block] 并返回 self (action 即 TraceBlock 实例) → 块内两次 print → 正常结束，exit 收到 (None, None, None)，打印 [exited normally]，返回的 None 被忽略。第二次 with：块内 raise TypeError → 异常机制把 (TypeError, 实例, traceback) 传给 exit → 打印 [propagating exception: <class'TypeError'>] → 返回 False → 异常被重新引发，一路传到进程顶层 → traceback 打印后程序以非零码退出。

·设计思想: return False 这一行是全例的题眼: 清理方法本身决定异常的生死。enter 返回 self 则演示了 as 变量与"管理器对象"的关系——协议允许它们相同 (self) , 也允许不同 (文件对象那种) 。示例自测代码 (main 块) 用"输出即文档"的方式把两条路径都演示了一遍。

34.9.3 Multiple Context Managers (多个上下文管理器)

英文原文

The with statement has one last card to turn over: it may also specify multiple (sometimes called "nested") context managers with comma syntax. For example, in the following code snippet that selects lines by substring, both files' exit actions are automatically run to close the files when the statement block exits, regardless of exception outcomes:

```
python with open('lines.txt') as input, open('matches.txt','w') as output:
    for line in input:
        if 'somekey' in line:
            output.write(line)
```

Any number of context manager items may be listed, and multiple items work the same as nested with statements. That is, the following hypothetical code:

```
python with A() as a, B() as b:
    statements
```

is equivalent to (and possibly simpler than) the follo

wing:

python with A() as a: with B() as b: statements

The net effect is that each context manager's entry and exit methods are run in turn on block entry and exit, and exceptions in the block are caught automatically and possibly reraised on with exit at the discretion of the outermost manager's exit method. Multiple file context managers, for instance, will all be run to open files on entry, and close them on exit—exception or not.

中文翻译

with 语句还有最后一张底牌：它也可以用逗号语法指定多个（有时称为“嵌套的”）上下文管理器。例如，在下面这段按子串筛选行的代码片段中，语句块退出时两个文件的退出动作都会被自动执行以关闭文件——与异常的结局无关：python with open('lines.txt') as input, open('matches.txt', 'w') as output: for line in input: if 'somekey' in line: output.write(line) 可以列出任意多个上下文管理器项，而且多个项的工作方式与嵌套的 with 语句相同。也就是说，下面这段假设性代码：python with A() as a, B() as b: statements 等价于（并且可能比）下面的写法更简单：python with A() as a: with B() as b: statements 净效果是：每个上下文管理器的进入与退出方法在块进入与退出时按顺序依次运行；块内的异常被自动捕捉，并可能在 with 退出时由最外层管理器的退出方法酌情重新引发。例如，多个文件上下文管理器：进入时全部运行以打开文件，退出时全部运行以关闭它们——无论有没有异常。

深度理解

- 核心概念：逗号分隔的多个 with 项 $\equiv$ 嵌套的 with——这是语法糖关系：with A() as a, B() as b: 精确等价于两层嵌套。进入顺序从左到右，退出顺序从右到左（先进入者后退出，类堆栈）。
- 底层实现：解释器按顺序执行：a = A().enter() $\rightarrow$ b = B().enter() $\rightarrow$ 块 $\rightarrow$ 退出时先 B().exit() 再 A().exit()。任一层 enter 抛异常，后面的项根本不会进入。这就是“资源获取按序、释放逆序”的 RAII 语义。
- 设计原因：多管理器让“同时处理两个资源”（如输入输出文件对、锁与连接）从嵌套三层缩进变成一行平铺——这正是作者说的“也许更简单”。
- 实际问题：with open(a) as inf, open(b) as outf: 是文件复制/过滤的标准写法；自 3.10 起还允许用括号跨行书写多管理器（34.9.4 会演示）。事务+锁、连接+游标等配对资源都可以一行搞定。
- 初学者误区：①以为多个 with 项是“并行”打开——不，是顺序进入、逆序退出；②错误地认为等价嵌套会改变异常传播（完全不变）；③忽略“若左侧项的 exit 返回真，右侧早已退出的管理器无法再干预”的细节。

代码分析

```

with open('lines.txt') as input, open('matches.txt', 'w') as ou...
    for line in input:
        #
        if 'somekey' in line:
            #
            output.write(line)
        #
# output input

#
with open('lines.txt') as input:
    #
    with open('matches.txt', 'w') as output: #
        for line in input:
            if 'somekey' in line:
                output.write(line)

```

·解释器如何处理：两个 open 的 enter 从左到右依次执行并绑定 input/output；块体照常运行。退出时以与进入相反的顺序调用两个 exit：先关 output（确保缓冲写盘），再关 input。若循环中抛异常，解释器仍先 output.exit（关闭文件、保证已写内容落盘）再 input.exit，异常继续传播。

·设计思想：“打开顺序与关闭顺序相反”是资源管理的普适定律——就像一次借两本书，还书时总是先还后借的。Python 用一条语句就把这条定律固化成了语法。

34.9.4 The Termination-Handlers Shoot-Out (终止处理器巅峰对决)

英文原文

You can find more info on context managers in Python's docs. Rather than getting more detailed here, let's close out this chapter with a quick look at this exte

nsion in action and a vetting of its roles. Using newer and redundant tools like with doesn't in and of itself prove intelligence, and it's important to understand the trade-offs such options imply.

First up, the following codes a parallel lines scan of files located in this book's examples package. It uses with to open two files at once and then reads and zips together their next-line pairs on each iteration of a for loop. Thanks to the file object's context manager, there's no need to manually catch exceptions or close files when finished:

```
text >> with open('lines1.txt') as file1, open('lines2.txt') as file2:
```

```
... for pair in zip(file1, file2): ... print(pair) ('hack\n','HACK\n') ('code\n','GOOD\n') ('well\n','CODE\n')
```

You might also use this coding structure to do a line s comparison of two text files. The following simply replaces the former's print with an if for a comparison operation, and adds an enumerate for automatic line numbers:

```
text >> with open('lines1.txt') as file1, open('lines2.txt') as file2:
```

```
... for (linenum, (line1, line2)) in enumerate(zip(file1, file2)): ... if line1.lower() != line2.lower(): ... print(f'{linenum} => {line1!r} != {line2!r}') 1 => 'code\n'!='GOOD\n' 2 => 'well\n'!='CODE\n'
```

That said, with isn't all that useful in the preceding examples when using CPython, because input file objects don't require a buffer flush, and file objects are closed automatically when garbage collected if still open. Moreover, exceptions in the with block are still propagated outside the statement if not explicitly caught. Hence, the temporary files would be auto-closed immediately and exception behavior would be the same for simpler code like this:

```
python for pair in zip(open('lines1.txt'), open('lines2.txt')): # Same if auto close print(pair) # Ditto for != test
```

On the other hand, some of the alternative Pythons of Chapter 2 may use different garbage collectors that require direct closes, to avoid taxing system resources. In addition, output files may require closes to ensure that any buffered content is transferred to disk so it's available for opens in later code.

The following lines filter code addresses both concerns, by automatically closing files on statement exit, exception or not (it also uses parentheses and line splits after with, available as of Python 3.10, and omits write counts for brevity):

```
text >> with (open('lines1.txt') as input,  
... open('uppers.txt','w') as output): ... for line in input:  
... output.write(line.upper()) >> print(open('uppers.txt').read()) # File content is available here HACK CODE  
WELL
```

Still, in simple scripts, we can often just open files in separate statements and close after processing if needed. There's no point in catching an exception if it means your program is out of business anyhow, and closes are required to flush output buffers only if files will be reopened by later code—which may never be reached after exceptions anyhow:

```
python input = open('lines1.txt') output = open('uppers.txt','w')
for line in input: # Same effect if files auto
close, output.write(line.upper()) # and file is not reopened ahead
```

Nevertheless, in programs that must both continue after exceptions and close output files for later use in the same program (or REPL) run, the `with` avoids an equivalent `try/finally` combination that may be more obvious to some readers, but also requires noticeably more code—eight lines instead of four, quantitatively speaking:

```
python input = open('lines1.txt') output = open('uppers.txt','w')
try: # Same but explicit close on errors for
line in input: output.write(line.upper()) finally: input.close()
# Ensure output file is complete output.close()
# Whether exception occurs or not
```

Even so, the `try/finally` is a single tool that applies to all finalization cases and makes code explicit. The `with` can be more concise for context-manager users, but applies only to objects that implement its complex

protocol, relies on implicit "magic" that obscures meaning, and adds redundancy that doubles the required knowledge base of programmers. As usual, you'll have to weigh these tools' trade-offs for yourself.

中文翻译

关于上下文管理器的更多信息可以在 Python 文档里找到。与其在这里讲得更细，不如用这个扩展的实战演示和对它角色的审视来结束本章。使用更新的、冗余的工具（如 with）本身并不能证明智慧，重要的是理解这些选项背后的权衡（trade-off）。第一个示例：对本书示例包中的文件做并行行扫描（parallel lines scan）。它用 with 一次打开两个文件，然后在 for 循环的每一次迭代里用 zip 把两文件的下一个行对读到一起并打印。多亏了文件对象的上下文管理器，完成后无需手动捕捉异常或关闭文件：

```
text >>> with open('lines1.txt') as file1, open('lines2.txt') as file2: ...
for pair in zip(file1, file2): ... print(pair) ('hack\n','HACK\n') ('code\n','GOOD\n') ('well\n','CODE\n')
```

你也可以用这种编码结构对两个文本文件做行比较（lines comparison）。下面只是把前者的 print 换成比较用的 if，并加上 enumerate 以自动编号：

```
text >>> with open('lines1.txt') as file1, open('lines2.txt') as file2: ...
for (linenum, (line1, line2)) in enumerate(zip(file1, file2)): ...
if line1.lower() != line2.lower(): ... print(f'{linenum} => {line1!r} != {line2!r}')
1 => 'code\n' != 'GOOD\n'
2 => 'well\n' != 'CODE\n'
```

话虽如此，在 CPython 下，with 在前面的例子里其实没那么有用：因为 input 文件对象不需要刷新缓冲区，而且文件对象在垃圾回收时若还开着会被自动关闭。此

外，with 块里的异常如果没有显式捕捉，照样会传播出语句。因此，对于这样更简单的代码，临时文件会立刻被自动关闭、异常行为也完全一样：python for pair in zip(open('lines1.txt'), open('lines2.txt')): # 若自动关闭则相同 print(pair) # 比较测试同理另一方面，第 2 章介绍的一些替代 Python (alternative Pythons) 可能使用不同的垃圾回收器，它们需要直接关闭以减轻系统资源负担。此外，输出文件可能需要关闭，才能确保缓冲的内容转移到磁盘、供后面的代码再次打开使用。下面的行过滤器 (lines filter) 代码同时解决了这两个顾虑：语句退出时自动关闭文件，异常与否都一样（它还用了 Python 3.10 起可用的 with 后括号+换行写法，并为了简洁省略了写入计数）：text >>> with (open('lines1.txt') as input, ... open('uppers.txt','w') as output): ... for line in input: ... output.write(line.upper()) >>> print(open('uppers.txt').read()) # 文件内容在这里已经可读HACK CODE WELL 不过，在简单脚本里，我们通常可以在独立的语句中打开文件，需要时处理完再关闭。如果捕捉异常意味着你的程序无论如何都要退出，那么捕捉它就毫无意义；而且只有“文件稍后会被后面的代码重新打开”时才需要关闭以刷新输出缓冲——而异常之后那段代码可能根本不会执行到：python input = open('lines1.txt') output = open('uppers.txt','w') for line in input: # 若文件自动关闭则效果相同，output.write(line.upper()) # 且文件不会被提前重开尽管如此，在那些“异常后必须继续运行、且要在同一次程序（或 REPL）运行中稍后使用输出文件”的程序里，with 可以避免一条等价的 try/finally 组合——后者对某些读者可能更直白，但代码量明显更多——量化地说，八行对四行：python input = open('lines1.txt


```
) output = open('uppers.txt','w') try: # 效果相同，但出  
错时显式关闭for line in input: output.write(line.upper()  
) finally: input.close() # 确保输出文件完整output.close()  
# 无论是否发生异常即便如此，try/finally 是一个适用于所有  
终结场景的单一工具，而且让代码显式。with 对上下文  
管理器的使用者来说可能更简洁，但它只适用于实现了其复  
杂协议的对象，依赖掩盖含义的隐式"魔法"，还引入了使程  
序员知识负担翻倍的冗余。像往常一样，你必须自己掂量这  
些工具的权衡。
```

深度理解

·核心概念：本小节是一场"工具辩论"：with vs try/finally vs "干脆不写"。作者用三个真实场景（行扫描、行比较、行过滤）逐一检验 with 的适用边界，结论：工具的价值取决于使用场景，盲目追新不等于聪明。

·底层实现：CPython 中文件对象在 del 时自动关闭（引用计数归零即触发），所以 zip(open(a), open(b)) 的临时文件在迭代结束时就被回收关闭——with 在这些只读场景里几乎只是"安心剂"。而输出文件带缓冲（buffered writer），不 close/flush 数据可能滞留缓冲区，这才是 with 真正立功的地方。

·设计原因：作者特意点出 with 的"隐性魔法与知识负担"：新程序员要理解协议、返回值语义、吞异常规则；try/finally 则一目了然。语言提供冗余工具，选择权留给工程师——这正是 Python 哲学里"有一种显而易见的做法，但允许替代方案"的现实版。

·实际问题：决策树很实用——①只读文件、程序跑完就退出：直接 `zip(open(...))` 也行；②输出文件要稍后重读：必须 `with` 或 `try/finally`；③跨实现（PyPy 等 GC 时机不可控）：`with` 是唯一确定性保证；④异常后程序本就退出：别画蛇添足。3.10+ 的括号多行写法让 `with` 也不臃肿。

·初学者误区：①以为"用了 `with` 就万事大吉"——`exit` 默认重抛异常，块内异常照样传播；②以为"文件不开 `with` 就一定会泄漏"——CPython 引用计数会自动关，只是时机不确定；③在"反正马上崩溃"的脚本里也堆 `try/finally` 八行样板；④误以为 3.10 括号写法是 `with` 的新语义——只是语法允许换行而已。

代码分析

```
# 1CPython with ""
with open('lines1.txt') as file1, open('lines2.txt') as file2:
    for pair in zip(file1, file2):          # zip
        print(pair)
# ('hack\n', 'HACK\n') / ('code\n', 'GOOD\n') / ('well\n', 'CODE\n...

# 2print if + enumerate
with open('lines1.txt') as file1, open('lines2.txt') as file2:
    for (linenum, (line1, line2)) in enumerate(zip(file1, file2)):
        if line1.lower() != line2.lower():  #
            print(f'{linenum} => {line1!r} != {line2!r}')
# 1 => 'code\n' != 'GOOD\n' / 2 => 'well\n' != 'CODE\n'

# 33.10+
with (open('lines1.txt') as input,
      open('uppers.txt', 'w') as output):
    for line in input:
```

```

        output.write(line.upper())           #
print(open('uppers.txt').read())           #
# HACK / CODE / WELL

# 4 try/finally vs
input = open('lines1.txt')
output = open('uppers.txt', 'w')
try:
    for line in input:
        output.write(line.upper())
finally:
    input.close()                           #
    output.close()                           #

```

·解释器如何处理：场景 1、2 中，zip(file1, file2) 逐行惰性配对，for 结束时解释器退出 with 块、自动调用两个 exit 关闭文件。场景 3 里 with 后的括号只是把两个管理项跨行书写（3.10+ 语法糖，语义不变）；uppers.txt 的关闭让缓冲大写内容落盘，所以随后的 open('uppers.txt').read() 能读到完整内容——若没关闭，缓冲区内容可能丢失。场景 4 用 try/finally 达到同一效果，但“出错也关闭”是手写保证的。

·设计思想：作者把三个例子按“with 的必要性”递增排列：只读（最不必要）→ 比较（同前）→ 写输出（必要）。最后的八行对四行对比，就是提醒读者：简洁不总是免费的——with 的简洁建立在协议、魔法与知识之上，try/finally 的冗长换来的是直白与普适。这就是“巅峰对决”没有赢家、只有权衡的原因。

34.10 Chapter Summary (本章小结)

英文原文

In this chapter, we took a more detailed look at exception processing by exploring the statements related to exceptions in Python: try to catch them, raise to trigger them, assert to raise them conditionally, and with to wrap code blocks in context managers that automate entry and exit actions. Up to this point, exceptions may seem like a fairly lightweight tool (apart from the with protocol, that is). The most complex thing about them may be how they are identified—a topic the next chapter will address by showing how exception objects are made. As you'll see there, classes allow you to code new exceptions specific to your programs. Before we move ahead, though, let's work through the following short quiz on the basics covered here

中文翻译

在本章中，我们通过探索 Python 中与异常相关的语句，更细致地考察了异常处理：try 用来捕捉它们，raise 用来触发它们，assert 用来有条件地引发它们，而 with 用来把代码块包裹在自动执行进入与退出动作的上下文管理器里。到目前为止，异常看起来可能是一个相当轻量级的工具（with 协议除外）。关于它们最复杂的事情也许就是“它们如何被识别”——这正是下一章的主题：展示异常对象是如何制造

出来的。你在那里会看到，类允许你编写程序专用的新异常。不过，在继续前进之前，让我们先做一下关于本章基础的简短测验。

深度理解

·核心概念：一张清单收束全章：try（捕捉）、raise（触发）、assert（条件触发）、with（上下文管理器包装）。四者合起来就是 Python 异常编码的完整语法地图。

·底层实现：这四样工具在字节码层面对应四类机制：try 的异常表（exception table, 3.11+ 的跳转表）、raise 的 RAISEVARARGS 指令、assert 的条件跳转（-O 下消失）、with 的 enter/exit 调用序列——它们共用同一条异常传播管线。

·设计原因：作者把“异常如何被识别”留到第 35 章，是为了让本章的读者先专注语句语法，再接受对象模型——两个层次分离教学，符合“先会用、再理解”的认知顺序。

·实际问题：读完本章，你已经能读懂并写出生产级异常代码的 90%：try/except/else/finally 组合、raise/from 链、assert 纪律、with 协议。剩下的 10%（自定义异常类、异常对象内部）在下一章补齐。

·初学者误区：把“异常处理语法”与“异常对象模型”混为一谈——except X as e 里的 X 是什么、e 里有什么，这些“识别问题”正是下一章的地盘；本章先别急。

34.11 Test Your Knowledge: Quiz (测验: 检验你的知识)

英文原文

1. What is the try statement for? 2. What are the two common variations of the try statement? 3. What is the raise statement for? 4. What is the assert statement designed to do, and what other statement is it like? 5. What is the with statement designed to do, and what other statement is it like?

中文翻译

1. try 语句是干什么的? 2. try 语句的两种常见变体是什么? 3. raise 语句是干什么的? 4. assert 语句被设计来做什么? 它像哪条语句? 5. with 语句被设计来做什么? 它像哪条语句?

深度理解

·核心概念: 五个问题覆盖本章全部核心语句, 且第 2、4、5 问都要求"对比"作答——作者设计测验的意图是检验概念之间的异同, 而不是背诵语法。

·底层实现: 回答"像哪条语句"这种问题时, 答案都是"语法简写/替代品": assert 是条件 raise 的简写, with 是 try/finally 的对象化替代品——理解这一点比记语法更本质。

- 设计原因：开放式简答题而非选择题，逼你用自己的话组织知识——教-学相长的经典手法。
 - 实际问题：建议先合上书默写答案再对照下一节参考答案，检验自己是否真的“讲得清”。
 - 初学者误区：只看参考答案、不先自答——那样测验就失去了诊断价值。
-

34.12 Test Your Knowledge: Answers (测验：参考答案)

英文原文

1. The try statement catches and recovers from exceptions—it specifies a block of code to run and one or more handlers for exceptions that may be raised during the block's execution. 2. The two common variations on the try statement are try/except/else (for catching exceptions) and try/finally (for specifying cleanup actions that must occur whether an exception is raised or not). Despite these logically distinct roles, the except and finally blocks may be mixed in the same statement, so the two forms are really part of the single try statement. Even when mixed with except, though, the finally is still run on the way out of the try, regardless of what exceptions may have been raised or handled. In fact, the combined form is equivalent to nesting a try/except/else in a try/finally. 3. The raise statement

ent raises (triggers) an exception. Python raises built-in exceptions on errors internally, but your scripts can trigger built-in or user-defined exceptions too with raise. 4. The assert statement raises an AssertionError exception if a condition is false. It's similar to a conditional raise statement wrapped up in an if statement, and can be disabled with a -O command switch. 5. The with statement is designed to automate startup and termination activities that must occur around a block of code. It is roughly like a try/finally combination in that its exit actions run whether an exception occurred or not, but it employs an object-based protocol for specifying entry and exit actions and may reduce code size for context-manager users. Still, it's not quite as general, as it applies only to objects that support its protocol; try with finally clauses can handle more use cases.

中文翻译

1. try 语句捕捉异常并从异常中恢复——它指定一段要运行的代码块，以及一个或多个针对块执行期间可能引发异常的处理器。2. try 语句的两种常见变体是 try/except/else（用于捕捉异常）和 try/finally（用于指定无论是否引发异常都必须发生的清理动作）。尽管它们在逻辑上角色不同，except 与 finally 块可以在同一条语句中混用，所以这两种形式其实是同一条 try 语句的两个部分。不过，即使与 except 混用，finally 仍然会在离开 try 时运行——无论引发或处理了哪些异常。事实上，组合形式等价于把一条 try/except

t/else 嵌套在一条 try/finally 里。3. raise 语句引发（触发）一个异常。Python 在出错时会在内部引发内置异常，但你的脚本也可以用 raise 触发内置或用户自定义异常。4. assert 语句在条件为假时引发一个 AssertionError 异常。它类似于一条包在 if 语句里的条件式 raise，并且可以用 -O 命令行开关禁用。5. with 语句被设计来自动化必须在代码块周围发生的启动与终止活动。它大致类似于 try/finally 组合——因为无论是否发生异常，它的退出动作都会运行——但它采用基于对象的协议来指定进入与退出动作，对上下文管理器使用者来说可以缩减代码量。不过，它没那么通用：它只适用于支持其协议的对象；带 finally 子句的 try 可以处理更多的用例。

深度理解

·核心概念：参考答案把五问浓缩成五句话，其中第 2 题最长——因为它包含本章最重要的两个等价关系：①except 与 finally 可混用但角色分离；②组合形式 $\equiv$ 嵌套形式。第 4、5 题的“像什么”分别指向 if + raise 与 try/finally。

·底层实现：第 4 题答案里“可以用 -O 禁用”呼应了 debug 编译期常量的机制；第 5 题“只适用于支持其协议的对象”点明了 with 与 try/finally 在通用性上的本质差距。

·设计原因：参考答案刻意不引入下一章（自定义异常类）的内容——测验边界与本章范围严格对齐，避免超前。

·实际问题：第 2 题的完整作答（混用规则 + 嵌套等价）就是一张可随身携带的“异常语法速查卡”，答得出它，本章就真的过关了。

·初学者误区：背答案而不是推演——比如回答第 2 题时说不出"else 必须搭配 except"、第 5 题时说不清"exit 返回假会重抛异常"，说明理解仍有缺口。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景：
 - try/except/else/finally 组合是 Web 服务、批处理脚本的通用错误处理骨架；raise ... from 是三层库（ORM、RPC 网关、缓存层）保留根因的标准手法；with 是文件、锁、数据库游标等一切"有始有终"资源的默认打开方式。
 - 断言（assert）在测试与开发期配置校验中高频出现，但生产代码的输入校验永远用 if/raise ValueError——因为 -O 部署会删掉所有断言，安全边界不能押在断言语义上。
 - 异常链（cause/context/from None）在日志系统中是"根因分析"的关键数据结构：上报错误时保留整条链，事故复盘才不会断线。
 - 与其他语言（Java/C++）的区别：

维度	Python	Java	C++
异常是对象吗	异常必为类实例，可携带任意属性	Throwable 子类	任意类型均可 throw（实践中多用类）
裸重抛	raise 一行	throw; 一行	throw; 一行

链式上下文	cause/context/ from None	3.7+ 起用构造 函数原因链	无内置支持
资源清理	with (协议化) + try/finally	try-with-resou rces (Java 7+	RAII (析构函数 , 无 finally 概
手动清理开关	-O 可删除断言	assert 可用 -da 禁用	assert 可用 ND EBUG 宏禁用

Python 与 Java/C# 的"using/try-with-resources"趋同，说明"协议化资源管理"已是现代语言共识；C++ 的 RAII 则把清理绑定到对象生命周期，连异常都无需参与。

- Python 发展历史背景：字符串异常（Python 2.6 之前）被类异常取代，是异常体系从"简单消息"走向"结构化对象"的分水岭；with 语句源自 PEP 343（2005 年），灵感来自 C# 的 using 与 PEP 310 的早期草案；raise from 与异常链由 PEP 3134（2005 年）引入，而 except（第 35 章）是 PEP 654（Python 3.11）对异常分组的补完。as 变量的"退出即删除"策略是 2005 年起为打破 traceback 循环引用而设。

- 高级开发者需要掌握的相关知识：

- sys.excinfo()、traceback 模块（格式化异常链）、contextlib（contextmanager、ExitStack、suppress、redirect）。

- 3.11+ 的异常组（ExceptionGroup/except）与 3.12 的 BaseExceptionGroup 语义。

- 异常对象内部：args、traceback、withtraceback()、自定义 str 对错误消息的定制。

- 自定义上下文管理器在"事务/回滚"场景的经典实现，以及 @contextmanager 生成器形态的等价写法。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) ——异常编码是 Python 程序员的日常工具，本语法细节章是"从会用到用好"的必经关口；with 与异常链更是生产代码高频考点。

·应该掌握到什么程度：

1. 必须：默写 try -> except -> else -> finally 顺序与合法性规则；能说出 finally 在四种场景下的行为；能用 raise 三种形式完成"抛出、传播、重抛"。

2. 必须：with 的协议 (enter/exit 参数与返回值语义) 与"as 变量 ≠ 表达式结果"。

3. 熟练：raise ... from 显式链与隐式 context 链的显示差异；from None 的用途。

4. 理解：as 变量退出即删除的作用域怪癖；-O 对 assert 的编译期影响。

5. 能够：在三段式对比 (with vs try/finally vs 裸写) 里说出取舍。

·后续应该学习的内容：

·第 35 章 (异常对象)：类的继承树、自定义异常类、except 异常组——本章所有"下一章会讲"的伏笔都会在那里兑现。

·第 36 章 (异常设计)：嵌套异常处理器的设计权衡与性能考量。

·进阶实践：contextlib 全家桶、logging 与异常链的配合、错误边界 (error boundary) 设计模式。